TaxonVision-MLOps

End-to-End Architectural and Operational Strategy Report

Systems Engineering

Benjamin Förster

30 October 2026

Abstract

TaxonVision-MLOps establishes an automated, production-grade Machine Learning Operations platform for fine-grained biological species identification. Transitioning away from ad-hoc, notebook-bound model development, this architecture utilises a unified open-source toolchain (DagsHub, DVC, MLflow, Optuna) mapped to a hybrid CI/CD topology: GitHub Actions executes lightweight quality gates, whilst a self-hosted Jenkins controller schedules GPU-bound workloads as ephemeral Kubernetes pods on a dedicated bare-metal host. Model training, hyperparameter optimisation, and ONNX compilation are performed exclusively on this remote execution plane; the developer workstation serves solely for authoring and orchestration. To satisfy regulatory and domain-specific safety requirements, the platform enforces epistemic bounds via Split Conformal Prediction and Energy-Based Out-of-Distribution (OOD) detection, guaranteeing marginal coverage in production. This document details the end-to-end pipeline architecture, verification methodologies, and trade-offs governing system design, accompanied by an audit of the implementation status against the phased delivery roadmap.

Contents

Abstract i

Preface 1

1 Architectural Context and Operational Maturity 2
1.1 Data Topography and Open-Access Mandates . . . 2
1.2 The Operational Bottleneck in Biodiversity Monitoring . . . 2
1.3 The TaxonVision Operational Maturity Framework . . . 3
1.4 Tri-Plane Orchestration Matrix . . . 4
1.5 Remote-Only Execution Principle . . . 4
1.6 Normative Architecture and Implementation Status . . . 4

2 Physical Infrastructure and Data Topology 6
2.1 Compute Topology and Workload Allocation . . . 6
2.2 Verified Software Baseline of the Host . . . 7
2.3 GPU Partitioning and Memory Budget . . . 7
2.4 Access Topology and Edge Security . . . 7
2.5 Cryptographic Storage Economics . . . 7
2.5.1 Circumventing the Multi-Terabyte Storage Trap . . . 8
2.5.2 Bounded Exemplar Storage Mathematics . . . 8
2.5.3 In-Cluster Object Storage: SeaweedFS Architecture and Multi-Remote Topology . . . 8
2.5.4 In-Cluster Ephemeral Upload Cache: Redis Architecture and Split-Brain Elimination . . . 10
2.6 Consolidated Relational and Telemetry Data Topology . . . 11

3 Open-Source MLOps Toolchain 12
3.1 Content-Addressable Storage and Cryptographic Versioning . . . 12
3.2 Unified Management Plane . . . 13
3.3 Bayesian Hyperparameter Optimisation and Telemetry . . . 13
3.3.1 Intelligent Search Mechanics . . . 13
3.3.2 Search Space and Optimiser . . . 14
3.3.3 MLflow 3.x Interlock Architecture and Model Passports . . . 14
3.4 Training Dynamics and Convergence Control . . . 16
3.4.1 Modular Sibling Topology and Clean Code Governance . . . 16

4 Dual Pipeline Architecture 18
4.1 Asymmetric Iteration and Execution Decoupling . . . 18
4.2 Automated CI/CD Orchestration & Registry Promotion . . . 18
4.2.1 Two-Stage Distribution and Provenance Attestation . . . 20
4.3 Automated Training on the Main Branch: The Epistemic Diamond DAG . . . 21
4.4 Convergence at the Serving Boundary . . . 22
4.5 Low-Latency Interpretability: Forward-Hooked CAM and Decoupled Serving . . . 23
4.5.1 Mathematical Formulation and Linear Head Contraction . . . 23

4.5.2 Vision Transformer Spatial Projection Invariants . . . 23
4.5.3 Content-Addressed Ephemeral Image Caching . . . 24
4.5.4 Progressive-Disclosure Operator UI and Differential CAM . . . 24

5 Vision RAG & Multimodal Knowledge Graph 26
5.1 Ingestion Engine & Reference Exemplar Catalogue Curation . . . 26
5.2 Visual Entity Linking: Parent-Child Chunking . . . 27
5.3 Single-Stage Filtered HNSW in LanceDB . . . 27
5.4 Conformal-Driven Lazy Execution . . . 28
5.5 Multi-Modal Re-Ranking via Reciprocal Rank Fusion . . . 28
5.6 Multimodal Context Synthesis & VLM Diagnostics . . . 29
5.7 Operationalising Taxonomic Mutations in the MMKG . . . 29
5.8 End-to-End Latency & Resource Allocation . . . 30

6 Epistemic Safety & Active Learning 31
6.1 Energy-Based Out-of-Distribution Detection . . . 31
6.1.1 Orthogonal Epistemic Defense via Predictive Shannon Entropy . . . 32
6.1.2 Analytical Upper and Lower Energy Bounds . . . 32
6.1.3 Dynamic Energy Threshold Calibration and Configuration Invariants . . . 33
6.2 Spatial-Temporal Constraint Masking . . . 34
6.3 Distribution-Free Uncertainty via Split Conformal Prediction . . . 35
6.3.1 Calibration Protocol and Finite-Sample Quantile Estimation . . . 35
6.3.2 Prediction Set Construction and Marginal Coverage Guarantee . . . 35
6.4 Conformal Triage Routing & Active Learning Feedback . . . 36
6.4.1 Active Learning Acquisition via Diverse Gradient Embeddings . . . 36

7 Observability, Testing, Deployment 38
7.1 Containerisation and Multi-Stage Runtimes . . . 38
7.1.1 Multi-Stage Image Compilation . . . 38
7.1.2 Kernel-Level Process Isolation . . . 39
7.2 K8s Cluster Orchestration and Workload Routing . . . 39
7.2.1 Asymmetric Pod Architecture and Ingress Routing . . . 39
7.2.2 Self-Healing and Health Probing . . . 40
7.2.3 Horizontal Pod Autoscaling on Custom Telemetry . . . 40
7.3 Telemetry, Metric Exporters, Concept Drift Monitoring . . . 40
7.3.1 Prometheus Telemetry Instrumentation . . . 41
7.3.2 Prometheus Cluster Realisation and TSDB Storage Mechanics . . . 41
7.3.3 Two-Tier Hybrid Feature Drift Architecture . . . 42
7.3.4 Dual-Path Alertmanager Routing and Automated Feedback Loops . . . 44
7.4 Multi-Tier Verification and Testing Strategy . . . 45
7.4.1 Tier 1: Upstream API Mocking . . . 45
7.4.2 Tier 2: Object Storage and Pipeline Mocking . . . 45
7.4.3 Tier 3: Mathematical Invariant Testing . . . 46
7.4.4 Tier 4: Statistical Metric Selection and Anomaly Verification . . . 46
7.4.5 Tier 5: Codebase Health, Test Coverage, and Cognitive Complexity . . . 47
7.4.6 Tier 6: Persistent Bayesian Telemetry and Deployment Smoke Testing . . . 48

8 Hardware Optimisation and Memory Mathematics 49
8.1 Static and Dynamic Quantisation Architectures . . . 49
8.2 Marginal Coverage Guarantees in Conformal Prediction . . . 50
8.3 Memory Complexity of Hierarchical Vector Graphs . . . 51

9 Conclusion and Operational Synthesis 52
9.1 Phased Delivery Roadmap and Implementation Status . . . 52

Preface

An objective systems engineering analysis indicates that the reference architecture presented in the academic syllabus is fundamentally ill-suited for high-throughput computer vision workloads. It attempts to force stochastic, high-dimensional visual feature spaces into the rigid, tabular paradigms of early MLOps. Adopting such blueprints directly would engineer a system destined to fail under the physical and latency realities of production.

Several pedagogical simplifications in particular warrant critical reassessment. The syllabus suggests triggering GPU-bound training loops via a synchronous HTTP `/train_evaluate` endpoint on the web server. From a systems perspective, this architectural anti-pattern blocks the asynchronous event loop, starves concurrent prediction requests, and induces request timeouts under operational load. Furthermore, querying an external MLflow server over the network to fetch model weights dynamically during live `/predict` requests introduces severe network round-trip latency, violating sub-25 ms Service Level Objectives (SLOs). In addition, staging raw imagery directly inside a relational database incurs substantial storage overhead when scaled to global citizen-science datasets. Finally, relying on tabular drift monitors and AUROC classification gates is mathematically unsuited for power-law biological distributions; empirical precision-recall metrics (PR-AUC, AUPIMO) are mandatory to assess operational efficacy accurately.

Consequently, the TaxonVision-MLOps architecture supersedes these paradigms with a production-grade systems architecture prioritising mathematical rigour, deterministic latency bounds, and verifiable reproducibility.

Rather than overloading a monolithic web server, this report outlines a strictly decoupled dual-pipeline architecture. Storage bloat is eliminated by streaming raw TAR archives directly from AWS Open Data into memory via zero-copy Apache Arrow buffers, whilst relying on lightweight DVC Parquet manifests for cryptographic immutability. To guarantee the sub-25 ms latency SLO, dynamic network loading is rejected; instead, pre-loaded, statically calibrated INT8 ONNX models are deployed, compressing the memory footprint by 75% and leveraging hardware vector instructions.

Most importantly, the platform rejects uncalibrated softmax heuristics in favour of verified epistemic safety. Anomalies are filtered natively in feature space via Energy-Based Out-of-Distribution (OOD) detection. Tabular drift monitors are replaced by a two-tier hybrid architecture: real-time streaming telemetry evaluates the linear-time Wasserstein-1 distance across pre-normalised feature activations, whilst offline continuous integration gates enforce full-manifold Maximum Mean Discrepancy (MMD) across the latent representation space. Crucially, Split Conformal Prediction transforms raw point probabilities into dynamic prediction sets backed by a distribution-free 95% marginal coverage guarantee $(1 - \alpha)$. When conformal gating detects ambiguity, the system executes a Single-Stage Filtered HNSW vector traversal, utilising multimodal Vision RAG to synthesise a grounded differential diagnosis.

This report documents a Level 4 MLOps platform. It represents a mathematically grounded, production-grade blueprint for global biodiversity monitoring.

Chapter 1

Architectural Context and Operational Maturity

The transition from localised machine learning prototypes to global ecological monitoring platforms requires fundamentally re-evaluating how systems manage scale, uncertainty, and regulatory compliance. Accordingly, this chapter establishes the operational baseline for the TaxonVision-MLOps architecture. It formalises the scale of citizen-science data streams, identifies the failure modes of unmanaged workflows, and defines the phased Machine Learning Operations (MLOps) maturity framework required to sustain a production environment.

1.1 Data Topography and Open-Access Mandates

Global biodiversity portals such as iNaturalist and the Global Biodiversity Information Facility (GBIF) aggregate continuous streams of species observations worldwide. Combined, these ecosystems host an estimated 270 million raw observations (as of public registry reporting in 2024 – 2026), of which roughly 170 million have achieved "research grade" status through community validation. Processing this high-volume stream requires strict cryptographic and legal compliance safeguards right at the ingestion edge.

In order to enforce open-access mandates, the ingestion engine inspects incoming observation metadata headers prior to staging objects locally. A cryptographic licence filtering module evaluates record metadata against allowed open-access definitions, thereby enforcing strict inclusion filters for Creative Commons Zero (CC0), Creative Commons Attribution (CC-BY), and Creative Commons Attribution-NonCommercial (CC-BY-NC). Any incoming observation tagged with restrictive, no-derivative (`CC-BY-ND`), or copyleft/share-alike variants (`CC-BY-SA`) is explicitly dropped at the network boundary to eliminate viral licensing constraints on derived model weights. Furthermore, to comply with GBIF and iNaturalist scientific lineage requirements, an automated attribution preservation mechanism extracts observation GUIDs, contributor ORCIDs, and original licence headers, serialising them alongside downstream feature vectors in order to maintain absolute lineage traceability.

1.2 The Operational Bottleneck in Biodiversity Monitoring

Fine-grained biological species identification at a global scale represents a formidable computational challenge. Citizen-science platforms generate continuous, high-volume streams of unstructured visual observations across highly variable environments. Historically, translating these raw field photographs into verified ecological intelligence has been severely constrained by fragile, ad-hoc machine learning engineering.

When predictive models are trained in isolated Jupyter notebooks – a state defined as Level 0 (Manual Process) maturity – they already suffer from non-reproducible software environments, untracked parameter drift, and silent degradation under natural distribution shifts. In an operational setting, these isolated scripts break down entirely during the handoff between data science and deployment teams. This renders the system incapable of sustaining continuous updates or scaling to support thousands of distinct taxa.

1.3 The TaxonVision Operational Maturity Framework

To process citizen-science visual streams reliably, the TaxonVision-MLOps architecture enforces a structured paradigm shift. Rather than relying solely on traditional MLOps reference architectures, we adopt Google's foundational 3-level MLOps spectrum (Levels 0 – 2: Manual, Automated Pipeline, and CI/CD) and extend it into a tailored 5-Tier Operational Maturity Framework. By introducing Level 3 (Feedback & Active Learning) and Level 4 (Epistemic Safety), the platform bridges standard deployment automation with mathematical coverage guarantees required for ecological monitoring.

Table 1.1: TaxonVision-MLOps 5-Tier Operational Maturity Framework (Adapted from Google MLOps)

Maturity Level	**Infrastructure Configuration**	**Operational Focus**
0: Manual (Google Baseline)	Isolated interactive notebooks; manual state handoffs.	Localised algorithm prototyping.
1: Pipeline (Google Baseline)	Automated DVC DAGs; consolidated 4-DB PostgreSQL tracking; baseline feature extraction.	Continuous training baseline & dataset lineage.
2: CI/CD (Google Baseline)	GitHub Actions PR gates; Jenkins RTX 5070 Ti GPU pods; Airflow 3 LocalExecutor; ONNX graph export.	Rapid, repeatable code, data & model releases.
3: Feedback (TaxonVision Extension)	PostgreSQL prediction telemetry; two-tier drift (W_1 & MMD^2); Grafana webhook relay; Label Studio triage.	Continuous drift detection, automated retraining & active learning.
4: Safety (TaxonVision Extension)	Split Conformal Prediction ($1 - \alpha$); Energy-Based OOD; Stage 7 Four-Gate Promotion Verdict.	Distribution-free coverage guarantees & production safety.

The comprehensive architecture diagram presented on the following page serves as the visual synthesis of this maturity framework. It maps the end-to-end TaxonVision-MLOps deployment strategy, detailing the physical separation between the offline classification engine and the continuous taxonomic Multimodal Knowledge Graph (MMKG). Furthermore, it illustrates the critical convergence of these dual pipelines at the inference serving boundary, where epistemic safety gates and active learning feedback loops guarantee sustained operational reliability.

1.4 Tri-Plane Orchestration Matrix

A common anti-pattern in MLOps architectures is tool overlap – attempting to force a single CI runner to handle cron scheduling, or forcing a workflow orchestrator to execute bare-metal GPU builds. The TaxonVision-MLOps platform eliminates ambiguity by enforcing a strict **Tri-Plane Orchestration Matrix**, cleanly partitioning duties across three specialised engines:

1. **GitHub Actions (Static PR Verification & Release Plane):** Ephemeral cloud runners executing fast pre-merge verification: Ruff linting, MyPy strict type analysis across 70 source files, Open Knowledge Format (OKF) schema validation, license audits, cognitive complexity bounds (≤ 15), and fast pytest suites. Upon merge to `main`, it builds hermetic wheel distributions and multi-arch production container images published to GHCR.
2. **Apache Airflow 3 (Stateful Control Plane & Scheduling):** Deployed as an in-cluster control plane running `LocalExecutor` backed by PostgreSQL. Airflow is strictly reserved for stateful, cron-scheduled, and event-driven workflows: the hourly prediction drift monitor (`prediction_drift_monitor` DAG), automated retraining coordination (`model_retraining` DAG), and bidirectional synchronisation with the Label Studio human annotation workspace. Airflow executes zero heavy machine learning operations itself, maintaining an extremely light operational footprint without requiring CUDA drivers or Python virtual environment overhead in scheduler containers.
3. **Jenkins (Bare-Metal GPU Compute Plane):** Deployed on the local bare-metal host, Jenkins manages ephemeral Kubernetes worker pods with exclusive NVIDIA GeForce RTX 5070 Ti access. Jenkins executes the computationally intensive stages of the DVC pipeline: high-throughput Vision Transformer feature extraction, Optuna Bayesian hyperparameter sweeps, and Stage 7 dual holdout evaluation with the Four-Gate Model Promotion Verdict.

1.5 Remote-Only Execution Principle

A further design decision concerns the location at which computation takes place. The developer workstation is deliberately not provisioned for model training. It serves as an authoring, linting and orchestration plane, whereas every training run, hyperparameter search and ONNX compilation is executed on the bare-metal host described in Chapter 2. These jobs are dispatched by the CI/CD controller only after all quality gates have been passed. This principle has three consequences:

- **Reproducibility:** Each run starts from an ephemeral container and a frozen dependency lock file, so that the execution environment cannot drift between runs.
- **Auditability:** The cryptographic dataset hash recorded by DVC is logged to the experiment tracker together with the metrics of every run, which links each model version to the exact data state it was trained on.
- **Resource isolation:** Long-running GPU workloads cannot degrade the interactive responsiveness of the development machine, and they are scheduled alongside the serving workload under an explicit memory budget (Section 2.3).

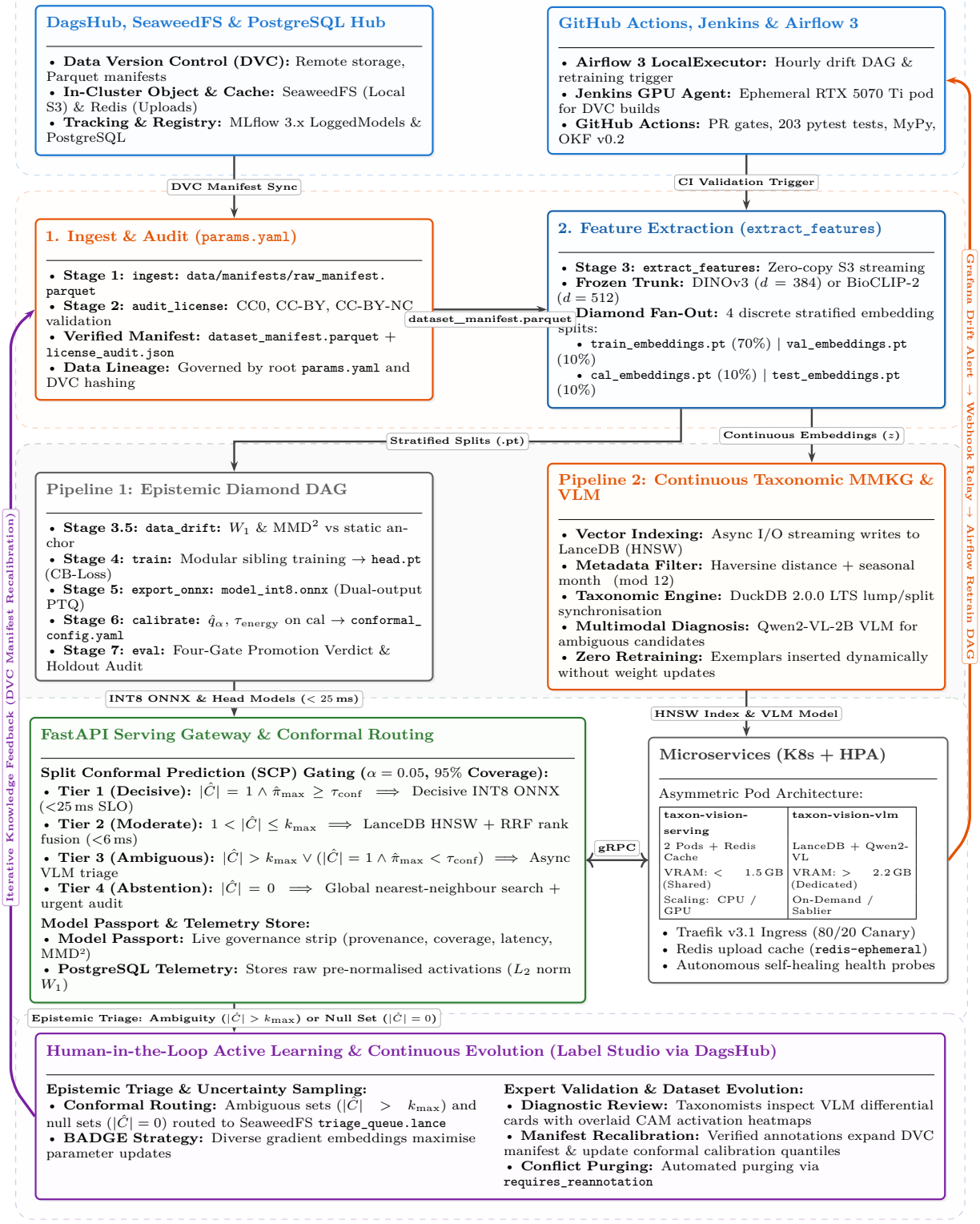
1.6 Normative Architecture and Implementation Status

This report distinguishes between the normative target architecture and the state of its implementation. Chapters 2 to 8 describe the target architecture and the mathematical properties it is required to satisfy. The implementation status verified against the code base, including known deviations and open items, is consolidated in Section 9.1 of Chapter 9. Numerical values given as design defaults, for example epoch budgets or batch sizes, are engineering guidelines and not measured results.

Production Pipeline Architecture

Decoupled Dual-Pipeline Engine · Epistemic Safety Gates · Hybrid Serving Boundary · Active Learning Closed Loop

Target: Bare-Metal Host (RTX 5070 Ti, i7-14700K) · Remote-Only Training
Epistemic Guarantee: Conformal Coverage $1 - \alpha \geq 0.95$
Inference SLO: Fast Path < 25 ms p95



Chapter 2

Physical Infrastructure and Data Topology

The deployment and continuous training of dense Vision Foundation Models (VFMs) require a balanced trade-off between infrastructure flexibility, execution predictability, and cost efficiency. While managed public cloud environments provide elastic scalability, sustaining persistent GPU instances and high-throughput data transfers for iterative research and continuous integration incurs prohibitive operational expenditures. To establish a cost-effective, self-contained development and execution plane, the platform deploys a dedicated on-premise compute node, combining hardware-level workload isolation with deterministic memory budgeting and transparent storage compression.

2.1 Compute Topology and Workload Allocation

Rather than provisioning continuous, multi-tenant cloud GPU infrastructure, the platform standardises on a dedicated bare-metal host engineered to isolate distinct computational regimes. This architecture amortises infrastructure costs while granting direct control over kernel process isolation, thread pinning, and storage caching.

The central processor features an Intel Core i7-14700K (20 cores, 28 threads: 8 Performance-cores and 12 Efficient-cores), supported by 128 GB of DDR5-5600 system memory. This hybrid core topology allows explicit segregation of heterogeneous workloads:

- **Mathematical Hot Loops:** Compute-intensive operations—such as classification head forward passes, PyTorch gradient calculations, and ONNX Runtime execution—are pinned strictly to the 16 Performance-core threads (`taskset -c 0-15`), preserving dedicated L2 and L3 cache lines.
- **Asynchronous I/O and Telemetry:** Concurrent background tasks—including Apache Arrow streaming, database indexing, Prometheus scraping, and HTTP ingress handling—are allocated to the 12 Efficient-core threads (`taskset -c 16-27`), eliminating thread context-switching on critical execution paths.

Neural network acceleration is provided by an NVIDIA GeForce RTX 5070 Ti graphics processor equipped with 16 GB of GDDR7 VRAM across a 256-bit memory bus. This provides sufficient Tensor Core throughput and memory bandwidth to execute local batch training, static INT8 post-training quantisation, and sub-15 ms inference validation without remote latency overhead.

High-throughput storage I/O is backed by a 2 TB PCIe Gen4 NVMe solid-state drive. To maximise read bandwidth when streaming millions of citizen-science image crops, the underlying BTRFS filesystem applies transparent `zstd:3` compression. However, because BTRFS utilizes Copy-on-Write (CoW) allocation by default, continuous database writes and Data Version Control (DVC)

chunk mutations induce significant block fragmentation. To guarantee deterministic read/write performance, all database backends and cache subvolumes are provisioned with the non-CoW attribute (`chattr +C`), ensuring contiguous physical block allocation across the NVMe bus.

2.2 Verified Software Baseline of the Host

The host (`hive-mind`) runs Ubuntu 26.04.1 LTS (codename *resolute*) on a LUKS-encrypted root volume. The proprietary NVIDIA driver in version 615.71.09 is installed, which reports a CUDA user-mode driver of version 13.4. Because the CUDA user-mode driver is backward compatible with older runtime libraries, container images are free to ship an older CUDA runtime. For the RTX 5070 Ti, which belongs to the Blackwell generation (compute capability 12.0), this runtime must however be CUDA 12.8 or newer: earlier PyTorch binary builds contain no kernels for this architecture and abort with a missing-kernel-image error. The CUDA runtime pinned in the dependency lock file is therefore a hard prerequisite of GPU execution rather than a cosmetic detail (see Section 9.1).

2.3 GPU Partitioning and Memory Budget

Consumer GPUs do not support hardware Multi-Instance GPU (MIG) partitioning. Without further measures, a Kubernetes pod requesting one GPU would therefore lock the entire device and starve every co-located workload. The cluster consequently uses the time-slicing mode of the NVIDIA Kubernetes device plugin, which advertises the single physical GPU as four virtual devices. Time-slicing multiplexes compute cycles but does not isolate memory: all slices share the same VRAM. Isolation is therefore enforced in software, by capping the fraction f of device memory available to the training process through the PyTorch per-process memory fraction. With $f = 0.7$ and a device memory of $M_{\text{VRAM}} = 16$ GB, the training budget evaluates to

$$M_{\text{train}} = f \cdot M_{\text{VRAM}} = 0.7 \times 16 \text{ GB} = 11.2 \text{ GB},$$

which leaves $M_{\text{VRAM}} - M_{\text{train}} = 4.8$ GB for the INT8 ONNX serving process and the display server. The latter reserve is comfortably larger than the serving footprint of ≤ 1.6 GB derived in Table 5.1. To guarantee that heavier secondary models do not violate this envelope, the auxiliary visual language model (`taxon-vision-vlm`) either scales to zero during active Jenkins GPU training windows or executes quantised VLM synthesis directly on the 16 host P-cores and 128 GB DDR5 system memory, maintaining the active GPU serving footprint strictly ≤ 1.6 GB VRAM.

2.4 Access Topology and Edge Security

The host exposes no inbound port to the public internet. Both the CI/CD controller and the operator-facing service are reached through outbound-initiated Cloudflare Tunnels, so that no router port forwarding and no dynamic DNS record is required. A Web Application Firewall rule at the Cloudflare edge restricts the controller hostname (`jenkins.benjamin-foerster.eu`) to authenticated GitHub webhook deliveries, whilst the operator dashboard and REST API are served live at `https://taxon.benjamin-foerster.eu` routing directly to Kubernetes NodePort 30080 behind an identity-aware Zero Trust access policy. Remote maintenance after a reboot is possible despite the encrypted root volume, because a minimal SSH daemon (Dropbear) in the initial ramdisk accepts the unlock passphrase before the root file system is mounted.

2.5 Cryptographic Storage Economics

A primary architectural constraint for open-source and low-budget MLOps is data storage costs. The platform standardises on DagsHub as the central management plane, which enforces a 20 GB free-tier storage limit on its DVC remote. This limit is fully viable, provided that the pipeline

enforces a strict physical separation between bulk raw observation streams and versioned metadata artefacts.

2.5.1 Circumventing the Multi-Terabyte Storage Trap

The raw citizen-science image corpora hosted on AWS Open Data registries collectively span tens of terabytes (an operational scale colloquially termed the "50 TB trap"). Mirroring this bulk volume locally or to a Git-tracked remote would be financially and computationally unviable. The system explicitly avoids this through zero-duplication streaming: ingestion pipelines stream raw TAR archives directly from AWS Open Data into memory via zero-copy Apache Arrow buffers.

Rather than storing the raw images, DVC tracks cryptographically validated Parquet index manifests. These lightweight tabular files contain observation IDs, DarwinCore taxonomic metadata, source URLs, and SHA-256 validation hashes. As an illustrative sizing calculation, an Apache Parquet manifest indexing 10 million distinct biological observations requires under 300 MB of storage due to columnar dictionary encoding.

2.5.2 Bounded Exemplar Storage Mathematics

For localised visual retrieval, the Vision RAG architecture curates a bounded Reference Exemplar Catalogue, constrained by policy to at most 50 validated environmental crops per species.

To demonstrate the storage envelope, we model an illustrative production scenario spanning 2,000 baseline taxa (yielding 100,000 curated images). Assuming an empirical compression target of approximately 45 KB per WebP crop (with a conservative 60 KB variance ceiling), the exemplar storage footprint evaluates as:

$$\text{Storage}_{\text{Exemplars}} = 100{,}000 \text{ images} \times 45 \text{ KB} = 4{,}500{,}000 \text{ KB} \approx 4.5 \text{ GB}$$

Furthermore, dynamic INT8 quantised ONNX model checkpoints typically range from 20 MB to 350 MB each depending on backbone parameter counts. Retaining a rolling buffer of production release candidates requires under 2.0 GB.

Under these illustrative assumptions:

$$\begin{aligned}\text{Storage}_{\text{Total}} = {} & 300 \text{ MB (Manifests)} + 4.5 \text{ GB (Exemplars)} \\ & + 2.0 \text{ GB (ONNX Models)} + 1.5 \text{ GB (DVC Churn)} \approx 8.3 \text{ GB}\end{aligned}$$

This 8.3 GB peak allocation (or 9.8 GB under peak 60 KB compression ceiling) operates well within the DagsHub 20 GB free-tier allowance, leaving ample headroom for active learning annotations.

2.5.3 In-Cluster Object Storage: SeaweedFS Architecture and Multi-Remote Topology

Whilst DagsHub acts as the external Git and DVC synchronisation mirror, high-throughput training and iterative inference loops cannot endure public internet round-trip latencies for ephemeral or high-volume intermediate data. Historically, early prototypes relied upon localised MinIO instances; however, architectural evolution and licensing constraints necessitated an absolute migration to **SeaweedFS** (`chrislusf/seaweedfs:latest`), licensed under the permissive **Apache 2.0** standard.

Licensing Imperatives and Community Sustainability

The containerised distribution of upstream MinIO transitioned to the Affero General Public License (AGPLv3), creating severe operational friction for downstream academic and commercial deployments. Following upstream Docker Hub repository purges, community continuity forks (such as Pigsty Silo) offer temporary respite. However, SeaweedFS eliminates all copyleft exposure through an unencumbered Apache 2.0 licence, ensuring complete sovereign control over data plane operations without legal ambiguity.

The Small-File Bottleneck and Haystack/Needle Mechanics

In deep vision pipelines, storage workloads exhibit extreme bimodal variance: Pipeline 1 produces heavy, multi-megabyte dense tensor matrices (stratified embedding splits under `data/embeddings/`, 50–500 MB), whilst Pipeline 2 requires rapid nearest-neighbour retrieval over $N = 100{,}000$ compressed visual exemplar crops ($s \approx 45$ KiB).

On conventional POSIX filesystems (such as standard ext4, XFS, or Copy-on-Write BTRFS), each individual file requires an independent inode allocation. Under continuous multi-worker streaming, reading N small objects introduces the high-latency phenomenon known as *disk-seek death*:

$$T_{\text{read}} = \sum_{i=1}^{N} \left(t_{\text{inode_lookup}}(i) + t_{\text{seek}}(i) + \frac{s_i}{B_{\text{NVMe}}} \right),$$

where $t_{\text{inode_lookup}} + t_{\text{seek}} \gg \frac{s_i}{B_{\text{NVMe}}}$, causing bus starvation and driving p95 retrieval latency beyond 30 ms.

SeaweedFS solves this structural bottleneck by adopting the Facebook Haystack needle-packing architecture. Individual observation crops are appended sequentially into contiguous, pre-allocated 32 GB volume files (`.dat`), whilst needle keys, physical byte offsets, and sizes are maintained within an in-memory compact hash index (`.idx`):

$$\text{Memory}_{\text{Index}} = N \times 16 \text{ bytes} \approx 100{,}000 \times 16 \text{ B} = 1.6 \text{ MB}.$$

Consequently, object lookups require exactly $O(1)$ disk seeks, enabling SeaweedFS to sustain sub-2 ms p95 read latencies on host NVMe storage under saturated 4 KiB random-read workloads.

Consolidated Cluster Daemon and Dynamic Bucket Provisioning

In Kubernetes, SeaweedFS is deployed as a consolidated, lightweight single-container daemon executing `weed server -dir=/data -s3 -s3.port=8333 -s3.config=/etc/seaweedfs/s3.json -volume.max=20`. This encapsulates four coordinated internal subsystems within an active memory envelope of approximately 80 MB RAM:

1. **The Master Service (port 9333):** Manages topology assignment, volume growth, and leader election.
2. **The Volume Server (port 8080):** Directly executes sequential needle read/write operations against the host NVMe filesystem.
3. **The Filer Engine (port 8888):** Maintains a hierarchical directory tree and metadata catalog backed by embedded LevelDB.
4. **The S3 REST Gateway (port 8333):** Translates AWS S3 Signature Version 4 (SigV4) API calls into high-performance Filer entries.

Unlike legacy MinIO configurations that mandate an auxiliary sidecar container executing asynchronous `mc mb` loops to initialise buckets, SeaweedFS dynamically auto-provisions bucket roots upon receiving valid authenticated S3 `CreateBucket` requests or pre-initialises buckets declaratively via a container lifecycle hook targeting the local Filer HTTP endpoint:

```
wget -q -O - --post-data="" "http://127.0.0.1:8888/buckets/taxon-dvc-cache/?op=mkdir"
wget -q -O - --post-data="" "http://127.0.0.1:8888/buckets/triage-queue/?op=mkdir"
```

Filesystem Invariants and Multi-Remote Routing

To prevent physical block fragmentation across the NVMe bus, the underlying persistent directory (`/var/lib/taxon-vision/seaweedfs`) is mounted onto a dedicated BTRFS subvolume configured with the `NOCOW` (`chattr +C`) attribute. Furthermore, because SeaweedFS volume daemons lock underlying volume data files (`.dat`), the Kubernetes Deployment enforces `strategy.type:`

`Recreate`. This prevents multi-attach volume lock collisions on single-node NVMe persistent volumes during rolling updates by guaranteeing that the terminating pod releases file locks before the replacement pod initialises. Automatic volume compaction runs in the background whenever deleted needle space exceeds 30%, reclaiming disk space without interrupting live inference loops.

DVC synchronisation is partitioned strictly across two distinct remotes:

- **The In-Cluster High-Throughput Remote (`local-seaweedfs`):** Targeting `s3://taxon-dvc-cache` at `http://seaweedfs.taxon-vision.svc.cluster.local:8333`. Dedicated to multi-gigabyte embedding caches (`train_embeddings.pt`, `val_embeddings.pt`, `cal_embeddings.pt`, `test_embeddings.pt`), enabling zero-egress, wire-speed caching for Bayesian hyperparameter optimisation sweeps.
- **The Public Lineage Remote (`origin`):** Targeting `s3://dvc` at DagsHub. Dedicated to release model checkpoints (`head.pt`) and INT8 quantised runtimes (`model_int8.onnx`).

2.5.4 In-Cluster Ephemeral Upload Cache: Redis Architecture and Split-Brain Elimination

While static assets and multi-gigabyte training tensors reside in SeaweedFS and DagsHub, the production serving layer introduces an ephemeral data plane requirement: holding raw uploaded image bytes across multi-step user interaction workflows.

Multi-Pod Split-Brain in Interactive Explainability

In the TaxonVision dashboard, an operator uploads an unlabelled organism crop to obtain conformal prediction sets via `POST /predict`. If the classification result exhibits epistemic ambiguity (e.g. $|\hat{C}| > 1$), the operator clicks "Explain CAM" in the web UI, which triggers a secondary request to `POST /explain` supplying the SHA-256 image hash.

In a horizontally scaled serving topology with two or more FastAPI pods behind Traefik ingress, an in-memory image cache (`OrderedDict` with threading locks) causes a catastrophic *multi-pod split-brain*:

1. **Pod 1** processes the initial `/predict` upload, storing the decompressed image array in its private process heap memory.
2. Traefik's round-robin or least-connections load-balancing algorithm routes the subsequent `/explain` call to **Pod 2**.
3. **Pod 2** inspects its local process RAM, fails to locate the hash, and returns HTTP 422 Unprocessable Entity ("Image not found or expired").

Redis Ephemeral Cache Implementation

To resolve this split-brain without imposing sticky sessions or heavy cloud object store latency, the cluster provisions an in-cluster **Redis Ephemeral Cache** (`redis-ephemeral:6379`). The cache architecture enforces strict resource and lifecycle bounds:

- **Memory Envelope and Eviction:** Memory is capped at 64 MB with `maxmemory-policy allkeys-lru`. Because each raw JPEG/WebP crop averages 45–60 KB, a 64 MB envelope comfortably buffers over 1,000 concurrent active operator sessions.
- **Deterministic Expiration:** Raw image bytes are written via atomic `SETEX` with a 900-second (15-minute) time-to-live:

  ```
  redis.setex(f"img:{image_hash}", 900, raw_bytes)
  ```

  This guarantees zero memory leak accumulation without requiring background thread garbage-collection loops in Python.

- **Sub-Millisecond Retrieval:** Any serving pod can retrieve the cached image payload in sub-millisecond latency ($t_{\text{redis}} < 0.5$ ms over localhost loopback/cluster SDN), enabling Pod 2 to compute Grad-CAM overlays seamlessly.
- **Graceful Degradation:** If the Redis service becomes temporarily unreachable, the `UploadCache` component automatically falls back to local in-memory caching and logs a structured warning, ensuring service resilience during cache restarts.

2.6 Consolidated Relational and Telemetry Data Topology

Early system prototypes fragmented state persistence across multiple disconnected SQLite databases (`mlflow.db`, `optuna_studies.db`, local telemetry logs). Under high-frequency concurrent traffic – such as parallel Bayesian hyperparameter trials running concurrently with live inference telemetry writes and Airflow scheduler heartbeats – SQLite's file-level locking mechanism inevitably induced write-lock contention (`database is locked`) and transaction timeouts on host NVMe storage.

To establish enterprise ACID transaction durability without introducing unwieldy operational overhead, the platform consolidates all relational and time-series metadata into a single, high-performance **PostgreSQL 16** cluster (`postgres:16-alpine`) hosted in-cluster. Operating with an active memory footprint of less than 80 MB RAM (dispelling the misconception that PostgreSQL imposes an intolerable 1.0 to 2.0 GB overhead for single-node deployments), the cluster establishes four strictly isolated logical databases provisioned during initialization:

1. **`mlflow`:** Serves as the primary metadata tracking backend for the self-hosted MLflow 3.x tracking proxy server, managing experiment runs, first-class `LoggedModel` entities, 4-split dataset lineage records, and logged metric parameters.
2. **`airflow`:** Backs the Apache Airflow 3 control plane under the `LocalExecutor`, maintaining stateful DAG definitions, task instance execution states, and connection credential vaults.
3. **`telemetry`:** Houses the production prediction telemetry store (`service/telemetry_store.py`). High-frequency inference events are recorded sequentially into the `predictions` table, persisting client coordinates, Linnaean taxonomic predictions, conformal prediction set sizes ($|\hat{C}(X)|$), Helmholtz free energy scores ($E(x)$), and unconstrained pre-normalised 768-dimensional latent embedding vectors ($\tilde{z}$) required for streaming drift audits.
4. **`optuna`:** Provides centralised persistence for Bayesian hyperparameter studies (`models/training/tuning.py`). To ensure driver compatibility across container boundaries, the application leverages modern `psycopg` (v3) while connecting the official Optuna Dashboard (`ghcr.io/optuna/optuna-dashboard:latest`) via the bundled `postgresql+psycopg2://` dialect.

To guarantee continuous physical block allocation and prevent write amplification across the host NVMe SSD, the persistent PostgreSQL database cluster volume (`/var/lib/taxon-vision/postgres`) is initialized on a dedicated BTRFS subvolume with the non-Copy-on-Write attribute applied (`chattr +C`). Crucially, to maintain developer agility and offline testing parity, SQLite is retained strictly as an automatic fallback: when `TAXON_DATABASE_URL` is absent or PostgreSQL is unreachable in local test environments, services automatically degrade to isolated SQLite WAL files without throwing unhandled connection errors.

Chapter 3

Open-Source MLOps Toolchain

In order to construct a production-grade machine learning platform without incurring prohibitive enterprise cloud licensing costs, our architecture integrates a curated selection of open-source components. However, merely assembling disparate tools is insufficient. True operational resilience requires a deep, mechanical interlocking between data versioning, experiment telemetry, and hyperparameter optimisation.

3.1 Content-Addressable Storage and Cryptographic Versioning

Standard version control systems, such as Git, are engineered to track line-by-line code modifications within directed acyclic graphs of text files. Applying this paradigm to binary biological datasets is fundamentally unsuitable. Committing a 50-gigabyte archive of high-resolution images forces the repository database to duplicate entire assets upon the slightest modification. This inevitably leads to memory exhaustion and repository collapse.

The system safely circumvents this limitation by utilising Data Version Control (DVC), which implements content-addressable storage. Rather than tracking the physical image matrices, DVC computes a unique SHA-256 cryptographic hash based strictly on the pixel content of the target directory. The heavy binary payloads are routed directly to a remote storage bucket, whilst a lightweight pointer file (for example, `dataset.dvc`) containing only the resultant hash and file metadata is generated.

This pointer file is subsequently committed to Git. When a deployment runner or data scientist executes a checkout command for a specific historical commit, the DVC client reads the hash from the pointer file, resolves the asset location in the remote cache, and constructs near-instantaneous filesystem reflinks (or hardlinks) into the local workspace. This dual-layered approach reliably decouples the heavy binary storage from the codebase, whilst guaranteeing mathematically verifiable, bit-for-bit data reproducibility.

To ensure non-blocking, rate-compliant upstream data acquisition from public biological APIs (such as iNaturalist and GBIF), ingestion workers enforce strict concurrency controls. API queries are mediated by an asynchronous token-bucket rate limiter (`aiolimiter.AsyncLimiter(1.0, 1.0)`), guaranteeing compliance with upstream throughput ceilings (maximum 1 request per second). Concurrent disk writes and manifest mutations across parallel workers are protected by POSIX advisory file locking via `filelock`, completely eliminating race conditions and corrupted Parquet manifests during multi-threaded data ingestion.

3.2 Unified Management Plane

Operating a distributed MLOps toolchain traditionally requires provisioning disparate enterprise infrastructure: an external PostgreSQL database cluster for experiment telemetry, dedicated S3 storage buckets, and complex Identity and Access Management (IAM) matrices to broker inter-service communication.

To eliminate cloud egress fees and maintain complete sovereign control over telemetry, the platform consolidates tracking and versioning into an in-cluster control plane. Telemetry and experiment metadata are anchored in the self-hosted **PostgreSQL 16** cluster within the dedicated `mlflow` database, operating with an active footprint of under 80 MB RAM. Heavy binary model checkpoints and evaluation artefacts are managed by a self-hosted MLflow 3.x tracking proxy server (`ghcr.io/mlflow/mlflow:v3.16.0`) executing with `-serve-artifacts` backed by the in-cluster SeaweedFS Apache 2.0 object store (`s3://mlflow-artifacts/`). DagsHub acts as an external Git and DVC synchronisation mirror and hosted Label Studio workspace, while SQLite WAL persistence is retained strictly as an automatic fallback for offline workstation iteration.

3.3 Bayesian Hyperparameter Optimisation and Telemetry

Model convergence, latency minimisation, and epistemic reliability depend fundamentally on tuning discrete and continuous training hyperparameters. Rather than viewing hyperparameter configuration and experiment tracking as disjoint manual procedures, the architecture integrates Bayesian optimisation algorithms directly with distributed metric logging pipelines in order to form a self-contained optimisation loop.

3.3.1 Intelligent Search Mechanics

Neural network architectures require the optimisation of external hyperparameters, such as learning rates, weight decay penalties, and conformal confidence thresholds. Traditional Grid Search algorithms evaluate these spaces exhaustively, incurring a computational cost of $\mathcal{O}(S^D)$, where S represents the number of discrete steps evaluated across D hyperparameter dimensions. On a single-node bare-metal architecture, this geometric explosion renders hyperparameter tuning computationally unviable.

The platform replaces brute-force evaluation with Optuna, driven by the Tree-structured Parzen Estimator (TPE). TPE abandons blind search grids in favour of Bayesian conditional density modelling. Let y denote the objective validation Macro PR-AUC metric to be maximised, and let y^* represent a dynamically evaluated scalar quantile threshold separating the upper quantile of top-performing trials from the baseline distribution. TPE models the conditional parameter probability distribution $p(\theta \mid y)$ directly as

$$p(\theta \mid y) = \begin{cases} \ell(\theta) & \text{if } y \geq y^*, \\ g(\theta) & \text{if } y < y^*, \end{cases}$$

where θ denotes the evaluated hyperparameter vector, $\ell(\theta)$ is the non-parametric probability density function constructed over the parameter configurations of superior historical trials ($y \geq y^*$), and $g(\theta)$ represents the density function constructed over sub-optimal evaluations ($y < y^*$).

The acquisition algorithm iteratively samples candidate configurations that maximise the density ratio $\ell(\theta)/g(\theta)$. This guides the GPU to evaluate parameter domains mathematically correlated with elevated classification precision on long-tail taxa, whilst maintaining sufficient dispersion to escape local extrema.

In order to conserve hardware cycles, Optuna applies the Asynchronous Successive Halving Algorithm (ASHA) with a minimum resource allocation of three epochs and a reduction factor of $\eta = 2$. A trial is first evaluated after three epochs; at each subsequent rung, only the superior

half of the candidate configurations ($1/\eta$) is permitted to proceed, whilst inferior trajectories are terminated immediately.

To guarantee continuity across iterative training cycles and prevent redundant evaluations, Optuna studies are backed by persistent PostgreSQL storage (`postgresql+psycopg2://optuna:...@postgres:5432/optuna`) with study identifiers scoped deterministically by vision backbone (for example, `tuning_dinov3` or `tuning_bioclip2`) under `load_if_exists=True`. SQLite storage (`sqlite:///data/optuna/optuna_studies.db`) is retained strictly as an automatic fallback for offline development. This persistence layer enables subsequent tuning invocations to warm-start from prior historical parameter trials, progressively refining the non-parametric Parzen density estimates without discarding previous evaluations.

3.3.2 Search Space and Optimiser

The hyperparameter space comprises six dimensions, sampled deterministically via a fixed pseudorandom seed (Table 3.1). Each trial initialises a classification head consisting of feature-jitter regularisation, dropout, and a dense projection layer, trained via the AdamW optimiser under Class-Balanced Loss [3].

Table 3.1: Hyperparameter search space of the Optuna study.

Hyperparameter	**Distribution**	**Range and role**
Learning rate η_{lr}	log-uniform	$[10^{-4}, 10^{-1}]$; step size of the AdamW optimiser.
Weight decay λ	log-uniform	$[10^{-6}, 10^{-2}]$; L_2 weight regularisation penalty.
Dropout probability p	uniform	$[0.0, 0.5]$; feature dropout rate prior to projection.
Class-Balanced Loss β	uniform	$[0.99, 0.9999]$; effective sample re-weighting parameter.
Mini-batch size B	discrete	$\{32, 64, 128\}$; stochastic batch sample cardinality.
Feature jitter σ	uniform	$[0.0, 0.05]$; zero-mean Gaussian perturbation $\mathcal{N}(0, \sigma^2\mathbf{I})$.

3.3.3 MLflow 3.x Interlock Architecture and Model Passports

Whilst Optuna orchestrates the search mathematics, it inherently lacks the infrastructure to permanently archive model provenance across a distributed engineering team. MLflow bridges this gap, operating as the continuous flight data recorder for our training pipeline.

The integration is implemented directly within the objective function and the overarching DVC pipeline. The tuning session registers as an MLflow experiment on the tracking server, and each trial is executed within a nested child run. To avoid socket starvation and excessive network I/O during rapid hyperparameter sweeps, all epoch metrics – training loss, validation loss, Top-1 accuracy, Macro F1-score, and validation Macro PR-AUC – are aggregated in memory and submitted at trial completion or ASHA pruning using `MlflowClient().log_batch()`. The lifecycle of the primary run is governed by a deterministic context manager, guaranteeing that unexpected execution exceptions transition the run to a `FAILED` state rather than hanging perpetually.

MLflow 3.x First-Class LoggedModels and Cross-Stage Continuity

Modern MLOps practice in MLflow 3.x departs from legacy path-based model registries, elevating models to first-class `LoggedModel` entities identified by an immutable identifier (`m-...`). The architecture enforces four critical tracking invariants:

1. **Cross-Stage DVC Run Continuity:** Stage 4 (`train.py`) initialises the parent MLflow run and serialises its unique identifier to `reports/train_metrics.json:mlflow_run_id`. Rather than scattering pipeline outputs across detached runs, subsequent stages (`export_onnx.py`, `calibrate.py`, and `evaluate.py`) re-enter this active context using the RAII context manager `resume_parent_mlflow_run()`. This unifies PyTorch weights, ONNX graphs, conformal thresholds, and evaluation reports into a single, cohesive lineage record.

2. **Four-Split Dataset Lineage (`mlflow.data`):** Every training run explicitly logs its four stratified tensor partitions (`train`, `val`, `cal`, `test`) alongside their DVC MD5 cryptographic content digests, instance counts, and DarwinCore schema tags using `mlflow.log_input()`. This establishes cryptographically verifiable provenance between raw citizen-science images and deployed model weights.
3. **Dual-Output Tensor Signatures:** During model packaging, an explicit `ModelSignature` is inferred from genuine validation batch tensors via `infer_signature()`. The signature enforces type-safe schemas across dual output nodes: unnormalised logits ($\mathbb{R}^{B\times K}$) and penultimate feature embeddings ($\mathbb{R}^{B\times D}$).
4. **Un-Gated Model Pollution Prevention:** To prevent cluttering the Model Registry with ephemeral experimental checkpoints, unit test runs and Optuna child trials are strictly forbidden from creating registered models. Model registration is restricted exclusively to the terminal evaluation stage of the canonical DVC pipeline.

The Model Passport Governance Standard

To establish cryptographic provenance and ensure zero-downtime serving attestation, the terminal evaluation stage compiles a structured **Model Passport** (`reports/model_passport.json`). The passport acts as the cryptographic identity certificate for the candidate model, binding source code, dataset hashes, mathematical thresholds, and promotion verdicts into a single immutable document (Table 3.2). Serving replicas load the passport on boot, exposing its parameters via `GET /api/v1/model/passport` and asserting hardware invariants before serving live traffic.

Table 3.2: Model Passport Schema and Governance Fields (`reports/model_passport.json`)

Field	Type	Description & Audit Verification Role
`model_id`	String	First-class MLflow 3.x model identifier (`m-...`) bound to the registered artifact.
`mlflow_run_id`	String	UUID of the parent DVC pipeline MLflow tracking run.
`git_commit`	String	Verified Git commit SHA from which the candidate was compiled.
`dvc_data_hash`	String	Cryptographic content hash of the training manifest (`dataset_manifest.parquet`).
`backbone`	String	Vision Foundation Model architecture identifier (e.g. `vit_base_patch14_reg4_dinov2`).
`onnx_md5`	String	MD5 checksum of the dual-output quantized runtime (`model_int8.onnx`).
`int8_quantized`	Boolean	Flag indicating whether Integer-8 Post-Training Quantization was applied.
`conformal_alpha`	Float	Nominal significance error tolerance $\alpha \in (0, 1)$ (default $0.05 \implies 95\%$ coverage).
`q_hat`	Float	Finite-sample calibrated conformal non-conformity threshold.
`tau_ood`	Float	Dynamically calibrated Helmholtz free-energy out-of-distribution threshold.
`eval_macro_pr_auc`	Float	Empirical holdout Macro PR-AUC score achieved on `test_embeddings.pt`.
`p95_latency_ms`	Float	Measured steady-state p95 forward-pass latency on target CPU Performance cores.
`four_gate_verdict`	Object	Structured boolean outcomes across Macro PR-AUC, INT8 fidelity, coverage, and live exchangeability.

Crucially, every training execution renders and logs publication-grade evaluation artefacts directly into the MLflow run:

1. `visualizations/confusion_matrix.png`: A class-normalised confusion matrix validating individual taxonomic boundaries.
2. `visualizations/precision_recall_curves.png`: Per-class Precision-Recall trajectories annotated with individual Average Precision (AP) scores across all catalogued taxa.

3. `visualizations/energy_distribution.png`: The empirical validation Helmholtz free-energy distribution annotated with the calibrated epistemic decision boundary τ_{OOD}.

When hyperparameter tuning is triggered, diagnostic Optuna plots (`optuna_optimization_history.png` and `optuna_param_importances.png`) are synchronously attached. Furthermore, the calibrated Conformal Prediction thresholds (`config/conformal_config.yaml`) are packaged directly into the model directory, guaranteeing atomic deployments of neural weights alongside their epistemic safety parameters.

3.4 Training Dynamics and Convergence Control

Because the backbone is frozen and its embeddings are cached, a training epoch of the classification head reduces to a small number of matrix products. The principal risk is therefore not under-fitting but over-fitting to the rare classes of the long tail. Three mechanisms control this risk. All of them monitor the validation Macro PR-AUC, which is also the promotion metric of the registry:

1. **Best-weight restoration:** The head weights of the epoch with the highest validation Macro PR-AUC are cached and restored once training has ended, so that the returned head is the best head and not merely the last one.
2. **Early stopping:** Training terminates when the metric has not improved for five consecutive epochs.
3. **Learning-rate reduction on plateau:** The learning rate is halved when the metric has not improved for two epochs. Since this patience is shorter than that of early stopping, the optimiser receives at least one epoch at a reduced rate before training terminates.

As design defaults, the baseline epoch budget is set to 30 epochs and the mini-batch size is 32 for training on cached embeddings (`epochs: 30`, `batch_size: 32` in `params.yaml`). Where embeddings are extracted from images by a Vision Transformer on the GPU, a batch size of 32 images keeps the activation memory comfortably inside the budget of Section 2.3. The feature extraction pipeline (`taxon_vision.pipelines.extract_features`) extracts these representations directly from AWS S3, caching them to disk for subsequent head convergence (see Section 9.1).

3.4.1 Modular Sibling Topology and Clean Code Governance

To eliminate monolithic software decay and satisfy Rule 08 clean-code invariants, the training engine is decomposed into single-responsibility sibling modules within `src/taxon_vision/models/training/`, strictly banning nested package circularities and maintaining cognitive complexity under the threshold $\mathcal{C} < 15$:

- **`runner.py`:** Orchestrates the overarching training execution lifecycle, learning rate plateau scheduling, and early stopping assertions.
- **`loops.py`:** Implements the tight inner forward and backward training loops operating over pre-cached GPU embedding tensors without dynamic allocation overhead.
- **`embeddings.py`:** Manages PyTorch TensorDataset abstractions, stratified minibatch creation, and tensor shape validation.
- **`tuning.py`:** Encapsulates Bayesian hyperparameter objective functions, trial scoping, and ASHA pruning logic.
- **`tracking.py`:** Provides RAII-scoped context managers for atomic MLflow run lifecycle management, guaranteeing clean teardown upon runtime exceptions.
- **`checkpoint.py`:** Manages best-weight restoration and deterministic PyTorch state-dict serialisation.
- **`types.py`:** Enforces parameter encapsulation via frozen dataclasses (`TrainingConfig`, `TrainingMetrics`, `OptunaTrialResult`), completely eliminating untyped tuple blindness.

Data augmentation applied during embedding extraction must respect biological invariants. Horizontal reflection yields physically plausible specimens, because most organisms are bilaterally or radially symmetric about a vertical axis. Vertical reflection is prohibited: gravity and phototropism impose a canonical orientation, so that an inverted specimen would never occur in field imagery and would merely bias the learned spatial priors.

Chapter 4

Dual Pipeline Architecture

A common anti-pattern in machine learning deployments is the monolithic pipeline, where neural network training sequences and data indexing tasks are tightly coupled into a single execution graph. The TaxonVision-MLOps architecture explicitly rejects this model. Instead, as visualised in the comprehensive architectural overview following Chapter 1, it establishes a dual-pipeline topology that decouples the weight-centric classification engine from the data-centric Multimodal Knowledge Graph (MMKG). These pipelines maintain independent lifecycles, converging exclusively at the inference serving boundary.

4.1 Asymmetric Iteration and Execution Decoupling

The necessity for architectural decoupling stems directly from the fundamentally asymmetric nature of biological modelling operations.

Pipeline 1: The Classification Engine. The primary neural backbone (for example, an optimised BioCLIP-2 [5] or DINOv3 transformer) is computationally expensive to train but relatively static in production. Refining its weights using Class-Balanced Loss [3] requires intensive GPU Tensor Core utilisation, distributed gradient synchronisation, and static ONNX Post-Training Quantisation (PTQ) calibration. However, once an optimal checkpoint is compiled, the model architecture may remain unchanged for weeks or months.

Pipeline 2: The Taxonomic MMKG. Conversely, the underlying citizen-science data stream is highly volatile. The pipeline must ingest continuous drops of new iNaturalist research-grade observations. More critically, biological taxonomies are frequently subjected to scientific realignments – species are split into novel taxonomic branches or lumped together under unified nomenclature. Pipeline 2 is an I/O-bound vector-indexing workflow responsible for parsing DarwinCore Parquet manifests, extracting zero-shot organism crops, filtering open-source licences, and appending dense embeddings to the Hierarchical Navigable Small World (HNSW) graph.

Forcing Pipeline 1 to re-execute a heavy neural network training sweep merely because Pipeline 2 updated an exemplar image for a specific beetle species wastes immense computational resources. Furthermore, physically separating these domains enforces strict failure isolation. If a malformed DarwinCore archive from an upstream provider induces a schema violation, Pipeline 2 will halt securely. However, Pipeline 1 remains entirely unimpeded, preserving our ability to compile, test, and deploy critical hot-fixes to the classification logic.

4.2 Automated CI/CD Orchestration & Registry Promotion

In order to transition from manual experimentation to a Level 3 MLOps deployment, the independent pipelines must be strictly governed by Continuous Integration and Continuous Deployment (CI/CD) triggers.

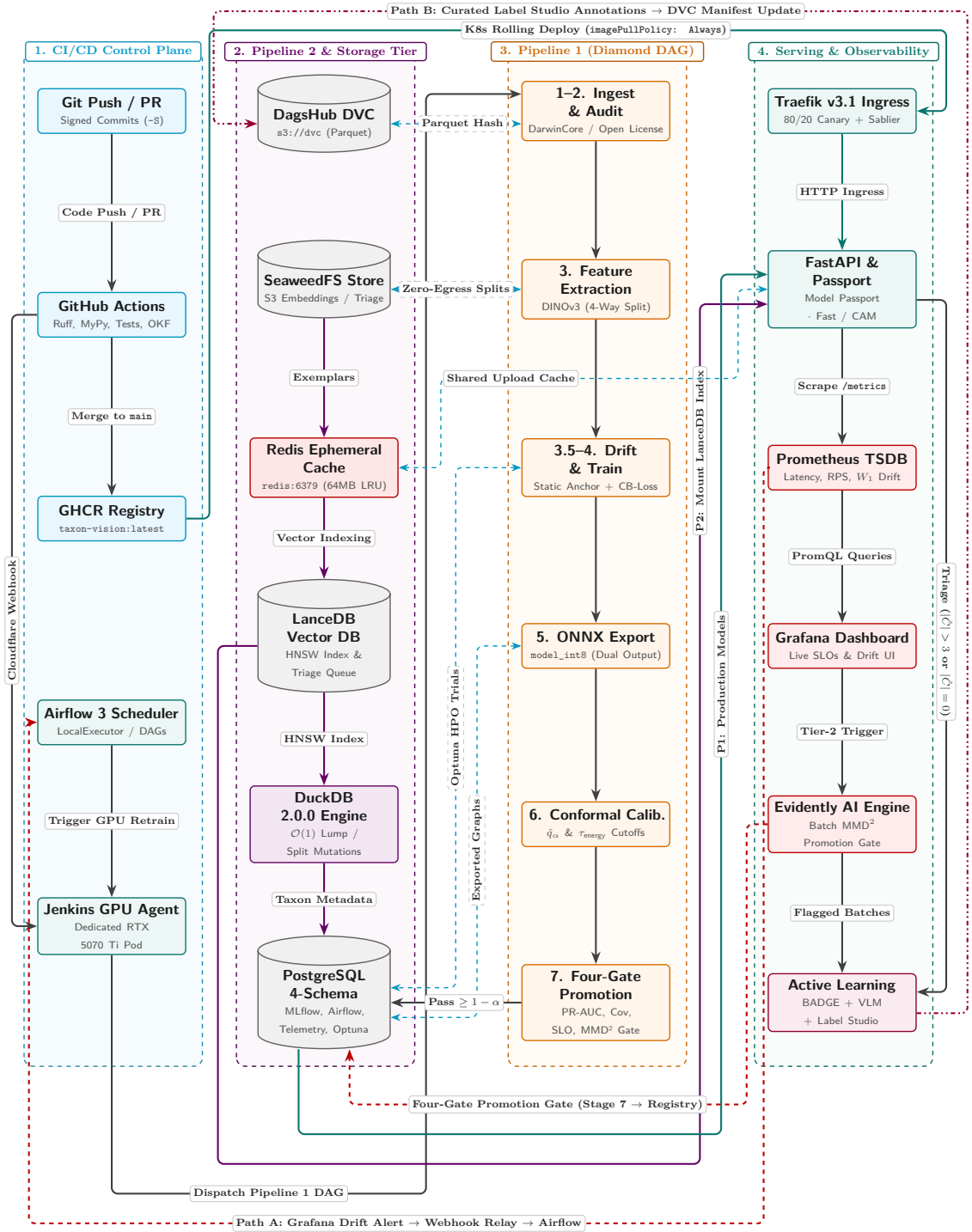


Figure 4.1: Decoupled Dual-Pipeline Architecture Flow: Pipeline 1 executes the 7-Stage Epistemic Diamond DAG with Stage 3.5 data drift auditing for neural classification head training, while Pipeline 2 manages asynchronous vector indexing in LanceDB and DuckDB 2.0.0 taxonomic mutations. Persistent telemetry, MLflow 3.x tracking, Airflow orchestration, and Bayesian trials are anchored in a consolidated 4-schema PostgreSQL cluster. Both pipelines converge at the FastAPI serving gateway (2 replicas with active Model Passport) behind Traefik v3.1 ingress and an in-cluster Redis ephemeral upload cache. Full-stack observability integrates Prometheus TSDB streaming metrics (W_1 drift), Grafana operational dashboards, Airflow 3 LocalExecutor drift and retraining DAGs, and closed-loop Active Learning feedback.

Two cooperating layers implement these triggers. When engineers push codebase modifications to the central repository, GitHub Actions executes lightweight static analysis (Ruff, MyPy), open-licence auditing, Open Knowledge Format (OKF) validation, unit and conformal test suites, and cognitive complexity scanning via `complexipy`. To validate serving endpoints during pull request review, the runner hydrates DVC model checkpoints directly from DagsHub. Upon merge to `main`, GitHub Actions automatically packages the production container image, hydrates promoted model checkpoints from DagsHub DVC, and publishes multi-tagged images to GitHub Container Registry (`ghcr.io/foersben/taxon-vision-mlops:latest`). In parallel, a signed GitHub webhook reaches a self-hosted Jenkins controller through an outbound-only Cloudflare Tunnel. The controller owns no persistent executors of its own: it schedules each build across a bifurcated, two-pod hybrid execution topology inside the k3s cluster of the bare-metal host:

1. **`jenkins-cpu-runner` (Lightweight Quality Gate):** Ephemeral CPU-only pod (1 core, 2 GB RAM) executing fast static verification: Ruff linting, MyPy strict typing across all 70 source files, 203 automated pytest tests, OKF frontmatter validation, and license audits. Pod execution employs a defensive `cleanWs` workspace guard to guarantee zero state leakage across builds.
2. **`jenkins-gpu-runner` (Heavy Compute Workload):** Dedicated GPU pod requesting an isolated NVIDIA GeForce RTX 5070 Ti allocation (16 GB VRAM) under `imagePullPolicy: IfNotPresent` to eliminate redundant multi-gigabyte container pulls over the WAN. This pod executes CUDA feature extraction, Optuna Bayesian hyperparameter sweeps, and INT8 post-training quantization.

Credentials for the tracking infrastructure, in particular the DagsHub access token, are held in the Jenkins credential store and are exposed to the build as environment variables for the duration of the run only. Production traffic is routed via Traefik v3.1 Ingress, supporting dynamic 80/20 canary traffic splits between `Production` and `Challenger` deployments, coupled with Sablier middleware for automated scale-to-zero resource management during idle operational intervals.

4.2.1 Two-Stage Distribution and Provenance Attestation

In addition to containerised cluster deployments, standard library distribution mandates a hardened Python wheel and source distribution pipeline. A dedicated two-stage GitHub Actions release workflow (`.github/workflows/release.yaml`) governs tagged releases (`v*`):

1. **Stage 1: Hermetic Package Compilation and Verification:** Executing within a dedicated CPU Pixi environment (`ci-dev`), the runner builds the distribution wheel and source tarball using `pip wheel -no-deps`. Crucially, the pipeline enforces strict dependency hygiene via `pip check`, validates runtime importing in an isolated environment, and computes cryptographic SHA-256 hashes across all build payloads.
2. **Stage 2: Release Publication and Cryptographic Attestation:** Upon successful gate completion, the workflow generates a formal GitHub Release, attaching the verified wheel, source archive, and SHA-256 checksum manifest to guarantee end-to-end supply chain provenance.

As the training pipeline initialises, it is specified to bind the MLflow telemetry to the exact Git commit that triggered the run, thereby ensuring cryptographic provenance between the source code and the resulting model weights:

```
1 mlflow.set_tag("git.commit", os.environ.get("GIT_COMMIT"))
```

Crucially, the CI/CD pipeline enforces an automated epistemic safety and promotion gate prior to deployment. Rather than promoting candidate weights immediately upon completing head training in Stage 4, the architecture reserves promotion authority exclusively to the terminal evaluation stage (Stage 7, `evaluate.py`). This eliminates a critical MLOps anti-pattern: Stage 4 only evaluates unquantised FP32 PyTorch weights on validation splits, remaining blind to INT8

quantisation drop, empirical test conformal coverage, and statistical exchangeability against live production traffic.

When a candidate clears all four gates of the Stage 7 Model Promotion Verdict (Section 7.3.3), the system programmatically registers the candidate as a first-class `LoggedModel` in the MLflow Model Registry. A qualified candidate that surpasses the incumbent `Production` model receives the alias `Challenger` pending canary deployment verification, whilst an inferior run is archived:

```
client.set_registered_model_alias(
    name="taxon_vision_classifier",
    alias="Challenger",
    version=mv.version,
)
```

4.3 Automated Training on the Main Branch: The Epistemic Diamond DAG

On the main branch, the CI/CD pipeline continues beyond static quality gates and executes the complete 8-stage Epistemic Diamond DAG on the bare-metal host. The stages are strictly governed by parameter definitions in the canonical root `params.yaml` and implemented as first-class CLI entrypoints within `src/taxon_vision/pipelines/`. The execution lineage forms a closed diamond topology that guarantees reproducible data-to-model provenance:

1. **Stage 1: Ingestion (`src/taxon_vision/pipelines/ingest.py`):** Queries research-grade citizen-science observation records across catalogued taxa via `INatAPIClient`, emitting the initial raw manifest `data/manifests/raw_manifest.parquet`.
2. **Stage 2: Licence Audit (`src/taxon_vision/pipelines/audit_license.py`):** Validates observation records against open-licence constraints (admitting only CC0, CC-BY, and CC-BY-NC), filtering non-compliant items and saving verified entries with full photographer attribution to `data/manifests/dataset_manifest.parquet` and `reports/license_audit.json`.
3. **Stage 3: Feature Extraction (`src/taxon_vision/pipelines/extract_features.py`):** Lazily streams observation imagery from AWS S3, passing images through the frozen foundation backbone (e.g., DINOv3 with $d = 384$, or BioCLIP-2 with $d = 512$). It partitions representations into four discrete, stratified tensor files under `data/embeddings/`: `train_embeddings.pt` (70%), `val_embeddings.pt` (10%), `cal_embeddings.pt` (10%), and `test_embeddings.pt` (10%). This diamond fan-out eliminates data leakage between training, calibration, and final holdout benchmarking.
4. **Stage 3.5: Training-Side Data Drift (`src/taxon_vision/pipelines/data_drift.py`):** Evaluates Tier 1 linear-time Wasserstein-1 (W_1) on pre-normalised embedding norms and Tier 2 multivariate Maximum Mean Discrepancy (MMD^2) between the freshly extracted training embeddings and the immutable reference baseline (`data/reference/reference_train_embeddings.pt.dvc`). If severe covariate shift is detected ($W_1 > \tau_{\text{drift}}$ or permutation $p < 0.01$), the pipeline fails fast before expending GPU compute on hyperparameter search and head training, serialising results to `reports/drift_metrics.json` and `reports/drift_report.html`.
5. **Stage 4: Head Training (`src/taxon_vision/pipelines/train.py`):** Trains a lightweight linear classification head on `train_embeddings.pt` and `val_embeddings.pt` using Class-Balanced Loss ($E_n = \frac{1-\beta^n}{1-\beta}$) and Bayesian hyperparameter sweeps via Optuna backed by PostgreSQL, outputting `models/checkpoints/head.pt` and `reports/train_metrics.json` with the active `mlflow_run_id`.
6. **Stage 5: ONNX Export & PTQ (`src/taxon_vision/pipelines/export_onnx.py`):** Compiles the PyTorch classifier into two specialised ONNX execution graphs:

(a) **End-to-End Dual-Output Graph (`models/checkpoints/model_int8.onnx`):** Ingests raw normalised image tensors ($\mathbb{R}^{3\times224\times224}$) and emits both unnormalised logits $f(x) \in \mathbb{R}^K$ and penultimate feature embeddings $z(x) \in \mathbb{R}^D$ in a single forward pass, quantised via INT8 Post-Training Quantisation (PTQ).

(b) **Dedicated Linear Head Graph (`models/checkpoints/head_int8.onnx`):** Ingests pre-computed embedding vectors ($z \in \mathbb{R}^D$) directly and emits logits $f(z) \in \mathbb{R}^K$. This enables sub-millisecond evaluation ($t_{\text{head}} < 0.2$ ms) during active learning query loops and vector triage without invoking the heavy Vision Transformer backbone.

7. **Stage 6: Split Conformal Calibration (`src/taxon_vision/pipelines/calibrate.py`):** Evaluates the holdout calibration partition `data/embeddings/cal_embeddings.pt` through `model_int8.onnx`. It calculates the empirical non-conformity quantile $\hat{q}_\alpha$ and Helmholtz free energy OOD threshold τ_{energy}, serialising runtime parameters to `config/conformal_config.yaml` and `reports/conformal_metrics.json`.

8. **Stage 7: Dual Holdout Evaluation & Four-Gate Model Promotion Verdict (`src/taxon_vision/pipelines/evaluate.py`):** Evaluates candidate backbones against the pristine holdout test partition `data/embeddings/test_embeddings.pt` using `model_int8.onnx` and `config/conformal_config.yaml`. It mathematically audits the Four-Gate Model Promotion Verdict: (1) Macro PR-AUC superiority, (2) INT8 quantization degradation $\leq 3.0\%$, (3) empirical marginal coverage bound ($\geq 1-\alpha$), and (4) p95 CPU latency under the SLO ceiling ($\leq 120\,\text{ms}$) with live traffic exchangeability (MMD^2 permutation $p \geq 0.01$). If all four gates pass, it promotes the model alias, writes `reports/pareto_evaluation.json`, and generates the cryptographic Model Passport (`reports/model_passport.json`).

Checkpoints, graphs, and evaluation reports are pushed to DVC remotes, and a rolling restart of the serving deployment (`imagePullPolicy: Always`) loads the fresh artefacts without interrupting service.

4.4 Convergence at the Serving Boundary

The two distinct pipelines exist in absolute isolation until the precise moment of user inference. The FastAPI serving application acts as the convergence node, simultaneously loading the INT8 quantised dual-output ONNX graph from Pipeline 1 into memory, whilst mounting the live HNSW vector index compiled by Pipeline 2.

When a field observation is submitted to the API, it is passed through the fast-path ONNX backbone. Emitting both class logits and dense feature embeddings in a unified forward pass eliminates a redundant second transformer evaluation, directly supplying $z(x)$ for spatial-temporal vector search and drift monitoring. To guarantee epistemic reliability whilst preserving strict latency bounds, the inference engine executes a deterministic 4-tier conformal routing contract governed by the prediction set size $|\hat{C}(X)|$ and top-1 probability $\hat{\pi}_{\max}(X) = \max_y \hat{\pi}_y(X)$:

1. **Decisive Classification ($|\hat{C}(X)| = 1$ with $\hat{\pi}_{\max}(X) \geq \tau_{\text{conf}}$):** The model exhibits high confidence and low epistemic ambiguity. When the top-1 softmax probability meets the operational threshold $\tau_{\text{conf}} = 0.60$, the endpoint dispatches the single predicted taxon immediately, fulfilling the strict sub-25 ms Service Level Objective (SLO) without activating retrieval overhead.

2. **Moderate Ambiguity ($1 < |\hat{C}(X)| \leq k_{\max}$, with $k_{\max} = 3$):** The conformal prediction set contains between two and three plausible sibling taxa. The serving engine executes a single-stage filtered HNSW traversal over LanceDB, constrained by the candidate class identifiers and spatio-temporal metadata, and performs Reciprocal Rank Fusion (RRF) over visual exemplar crops with less than 6 ms overhead to deliver a multi-candidate comparative differential to the operator.

3. **Severe Ambiguity or Under-Confident Singleton ($|\hat{C}(X)| > k_{\max}$ or ($|\hat{C}(X)| = 1 \wedge \hat{\pi}_{\max}(X) < \tau_{\text{conf}}$)):** The prediction set exceeds the bound of actionable human differential

review, or a singleton set fails the minimum confidence threshold ($\hat{\pi}_{\max} < 0.60$). The query is scheduled for asynchronous multimodal resolution via gRPC to the quantised visual language model (`taxon-vision-vlm`, based on Qwen2-VL-2B) and persisted locally via `persist_flagged_observation()` before ingestion into `triage_queue.lance` on SeaweedFS for expert review.

4. **Epistemic Novelty / Complete Abstention ($|\hat{C}(X)| = 0$):** The observed sample violates the conformal coverage boundary, indicating an out-of-distribution observation, unmodelled developmental instar, or extreme sensor degradation. The system executes complete epistemic abstention, bypasses automated classification entirely, executes an unconstrained nearest-neighbour retrieval over the global exemplar space, and dispatches an immediate priority triage alert to human taxonomists.

4.5 Low-Latency Interpretability: Forward-Hooked CAM and Decoupled Serving

Providing diagnostic transparency to human reviewers and field taxonomists requires visual interpretability tools such as Class Activation Mapping (CAM). Whilst gradient-based extensions such as Grad-CAM are standard in offline research, they require a computationally expensive backward pass during inference. Executing gradient backpropagation on a production server introduces an unacceptable 60–100 ms processing overhead, violates the sub-25 ms latency Service Level Objective (SLO), and prevents deployment in highly optimised, forward-only runtimes such as INT8 ONNX Runtime.

To achieve visual interpretability without violating strict operational latency constraints, the architecture executes forward-only Class Activation Mapping, tightly integrated into a decoupled, progressive-disclosure serving workflow.

4.5.1 Mathematical Formulation and Linear Head Contraction

The linear classification head $f(z) = Wz + b$ parameterises the mapping from an embedding vector $z \in \mathbb{R}^d$ to K unnormalised logits, where $W \in \mathbb{R}^{K \times d}$ denotes the weight matrix. During the initial forward pass, forward execution hooks capture unpooled spatial activation features $A \in \mathbb{R}^{d \times H_{\text{grid}} \times W_{\text{grid}}}$ emitted by the final backbone stage prior to global average pooling. Because the final classification head is linear, the importance weight w_k^c of feature channel k for a target class c is identically the learned weight matrix element $W_{c,k}$.

The class activation map $L_{\text{CAM}}^c \in \mathbb{R}^{H_{\text{grid}} \times W_{\text{grid}}}$ is synthesised through the linear combination

$$L_{\text{CAM}}^c = \text{ReLU}\left(\sum_{k=1}^{d} W_{c,k} A^k\right),$$

where ReLU discards negative spatial influences to isolate regions that positively support class hypothesis c. The resulting two-dimensional activation matrix is min-max normalised and bilinearly upsampled to the native input resolution $(H_{\text{img}}, W_{\text{img}})$. By relying exclusively on forward activations and static linear projection weights, this formulation synthesises high-fidelity spatial saliency heatmaps in under 2 ms on CPU, entirely preserving the forward-only, quantisation-friendly execution paradigm.

4.5.2 Vision Transformer Spatial Projection Invariants

Modern vision foundation models (such as DINOv2 `vit_base_patch14_reg4_dinov2` or BioCLIP-2) emit flattened token sequences rather than two-dimensional convolutional grids. Specifically, the unpooled feature tensor exhibits the shape $(B, 1 + N_{\text{reg}} + N, d)$, comprising a leading classification token (`[CLS]`), N_{reg} optional register tokens introduced to absorb high-norm artefacts, and N spatial patch tokens.

To prevent spatial misalignment or index corruption, the projection subroutine (`_project_vit_features` in `src/taxon_vision/inference/cam.py`) enforces strict architectural invariants:

1. **Prefix Token Stripping:** The leading $1 + N_{\text{reg}}$ non-spatial tokens are sliced off and discarded.
2. **Square Lattice Assertion:** The remaining token count N must satisfy a perfect square topology, $H_{\text{grid}} = W_{\text{grid}} = \lfloor \sqrt{N} \rfloor$ (e.g., $N = 196$ tokens emitted by a 224×224 input with patch size 14×14 yield an exact 14×14 spatial lattice). Any dimensional mismatch raises a typed `SaliencyExtractionError`.
3. **Tensor Reshaping and Einstein Summation:** The projected token tensor is reshaped to $(B, d, H_{\text{grid}}, W_{\text{grid}})$, after which channel contraction against class weight vector $W_{c,:}$ is evaluated via Einstein summation (`torch.einsum('bdhw,d->bhw', features, weights)`).

If feature extraction or spatial projection fails due to unsupported backbone pooling or corrupt input dimensions, the system safely catches the error and degrades gracefully without failing the surrounding HTTP transaction.

4.5.3 Content-Addressed Ephemeral Image Caching

Computing CAM saliency synchronously on every prediction request would unnecessarily consume CPU cycles and bloat response payloads for automated clients that only require categorical predictions. Conversely, forcing the client to re-upload raw multi-megabyte image payloads whenever an operator requests visual explanation incurs substantial network latency and memory overhead.

To resolve this trade-off, the serving layer implements a content-addressed ephemeral image cache governed by `ImageCacheProtocol` in `src/taxon_vision/service/storage.py`, backed by an in-cluster Redis ephemeral instance (`RedisImageCache`) with in-memory fallback (`MemoryImageCache`):

1. **Content Addressing via SHA-256:** Uploaded raw image bytes are hashed using SHA-256 to generate an immutable content digest (`image_hash`).
2. **Multi-Pod Split-Brain Mitigation:** Rather than isolating cached payloads within a single pod's heap, images are stored in a shared cluster Redis instance (`redis-ephemeral:6379`), ensuring that subsequent CAM attribution requests routed round-robin across replicas resolve the image payload seamlessly.
3. **Bounded LRU and Rolling TTL Eviction:** The cache enforces a strict memory ceiling (`maxmemory 64mb` with `allkeys-lru`) and monotonic rolling time-to-live eviction (`image_cache_ttl_seconds = 900.0`, or 15 minutes, refreshed upon access), configured via `config/default_config.yaml`.
4. **Zero Fast-Path Disk I/O:** Temporary images are never written to disk during the standard prediction path, avoiding unbuffered file I/O latency on high-throughput serving nodes.

4.5.4 Progressive-Disclosure Operator UI and Differential CAM

The FastAPI serving runtime and HTMX dashboard operationalise this decoupled architecture through progressive disclosure:

1. **Fast-Path Inference (< 15 ms):** The synchronous prediction endpoints (`POST /ui/predict-htmx` and `POST /api/v1/predict`) execute the dual-output ONNX model, evaluate conformal prediction sets $\hat{C}(X)$, and compute epistemic telemetry (Shannon entropy and Helmholtz free energy compared against thresholds τ_{OOD} and τ_{entropy}). The dashboard immediately renders the prediction card containing candidate probability bars and epistemic telemetry badges without waiting for saliency synthesis.
2. **On-Demand Lazy Heatmap Synthesis (`/api/v1/explain`):** The CAM interface is embedded in a collapsible HTML5 `<details>` drawer. The client browser triggers an asynchronous HTMX request (`hx-post="/api/v1/explain?image_hash=...&target_class_`

`index=...`") with the trigger condition `hx-trigger="toggle once"`, synthesising the heatmap overlay only when the operator actively opens the drawer.

3. **Differential Multi-Candidate Inspection:** When an observation yields a non-singleton conformal prediction set (e.g., $|\hat{C}(X)| \in \{2, 3\}$), the drawer provides interactive candidate toggles. Operators can click individual competing taxa to dynamically inspect class-specific activation maps for contrasting hypotheses on the identical cached image. This allows taxonomists to verify whether the model attended to genuine diagnostic morphology (e.g., wing venation patterns or floral stamen structures) or spurious background correlation.
4. **Fault-Tolerant Rendering:** If an expired hash or unsupported architecture triggers an exception during explanation, the endpoint returns an inline warning fragment (`partials/cam_error.html`) at HTTP 200 for HTMX clients (and typed HTTP 422 for machine API consumers), ensuring that the operator dashboard remains fully functional and responsive.

Chapter 5

Vision RAG & Multimodal Knowledge Graph

Standard text-centric Retrieval-Augmented Generation (RAG) paradigms typically rely on document chunking, token embedding, and semantic similarity search in order to ground large language models. In visual ecology, we must fundamentally transform this concept into multimodal equivalents. Citizen-science platforms aggregate massive volumes of environmental imagery, but raw field photographs are highly dense, noisy, and unstructured. Rather than attempting to compute and search vector embeddings across the entire 170-million-image iNaturalist corpus [6], the TaxonVision-MLOps architecture couples a curated Reference Exemplar Catalogue with single-stage spatio-temporal filtered graph traversal in LanceDB and conformal-driven lazy execution.

5.1 Ingestion Engine & Reference Exemplar Catalogue Curation

A naive approach to visual retrieval attempts to pass every available field photograph through a deep Vision Foundation Model (VFM) in order to construct an exhaustive vector database. In the context of global biodiversity monitoring, this brute-force strategy requires generating dense embeddings for more than 170 million observations. Storing 768-dimensional 32-bit floating-point (FP32) representations for this volume would require over 520 GB of addressable memory purely to hold the vector matrices in RAM, alongside years of continuous GPU compute on workstation-grade silicon.

The platform actively circumvents this computational barrier by curating a deterministic Reference Exemplar Catalogue. Instead of tracking all historical user submissions, our ingestion pipeline filters raw DarwinCore observation records according to four strict quality predicates:

1. **Validation Tier:** Observations must hold official `research_grade` status, confirming agreement across three or more independent community experts.
2. **Licence Compliance:** Records must possess open-access metadata headers matching Creative Commons Zero (`CC0`), Creative Commons Attribution (`CC-BY`), or Attribution-NonCommercial (`CC-BY-NC`). Commercial, no-derivative (`CC-BY-ND`), or copyleft/share-alike variants (`CC-BY-SA`) are explicitly dropped at the ingestion boundary to eliminate viral licensing constraints on derived model weights.
3. **Signal Quality:** Raw candidate photographs must exceed a minimum resolution of 1024×1024 pixels in original source repositories for exemplar inclusion, whilst the streaming ingestion pipeline pulls 500px medium locators for fast Arrow serialisation. Furthermore, upstream zero-shot object detection must isolate the primary organism with a detection confidence score greater than 0.85.

4. **Diversity Sampling:** The catalogue selects up to 50 exemplar observations per taxon. In order to capture seasonal and geographical morphological variations, observations are partitioned into discrete spatial clusters using Uber H3 hexagonal spatial cells at Resolution 4.

By bounding the catalogue to 2,000 baseline taxa with 50 exemplars each, the vector database maintains an upper bound of 100,000 reference vectors. The total memory required to hold these 768-dimensional FP32 vectors in RAM evaluates to

$$\text{Memory}_{\text{Vectors}} = 100{,}000 \times 768 \times 4 \text{ bytes} \approx 307.2 \text{ MB}.$$

Stored as compressed WebP image crops averaging 45 KB each, the total exemplar disk image footprint is approximately 4.5 GB. This architectural decision allows the entire retrieval index to reside in high-speed host RAM, completely eliminating disk I/O bottlenecks during retrieval.

5.2 Visual Entity Linking: Parent-Child Chunking

In text-based RAG architectures, feeding an entire multi-page document into an embedding model dilutes key semantic signals. Similarly, passing an uncropped field photograph into a Vision Foundation Model introduces severe environmental noise. Background clutter – such as soil, sky, vegetation, water reflections, or human hands holding a specimen – dominates the receptive field, creating spurious cosine similarities between taxonomically distant species that inhabit similar macro-environments.

In order to isolate diagnostic visual signals, the system implements Parent-Child Visual Chunking. An upstream zero-shot detector (YOLO-World or OWL-ViT v2) scans the raw photograph to isolate the target specimen, extracting a normalised bounding box $B = [x_{\min}, y_{\min}, x_{\max}, y_{\max}]$. The organism crop ($I_{\text{child}}$) is scaled to 224×224 pixels and forwarded to the frozen vision backbone (BioCLIP-2 or DINOv3) to produce a unit-normalised visual embedding vector $z_{\text{child}} \in \mathbb{R}^d$ ($d = 384$ or 512):

$$z_{\text{child}} = \frac{f_{\text{VFM}}(I_{\text{child}})}{\|f_{\text{VFM}}(I_{\text{child}})\|_2},$$

where f_{VFM} represents the forward pass of the frozen vision backbone. The original uncropped field photograph (I_{parent}) is preserved as an environmental habitat context layer and cryptographically linked to the child crop via the relational manifest tuple:

$$\text{EntityLink} = \langle \text{CropID}, \text{ParentURI}, B, \text{DarwinCorePayload} \rangle .$$

During vector search, similarity queries execute strictly over child embeddings (z_{child}); upon retrieval for multimodal reasoning, both the child crop and parent context are presented to the evaluator.

5.3 Vector Database: Single-Stage Filtered HNSW in LanceDB

Biological retrieval systems face a severe domain-specific failure mode known as *convergent evolution*. Under similar ecological pressures, unrelated organisms across distinct continents frequently evolve nearly identical visual morphologies. For example, the European Hornet (*Vespa crabro*) and the North American Cicada Killer (*Sphecius speciosus*) exhibit strikingly similar body geometry and colouration patterns.

If a retrieval engine implements standard two-stage post-filtering – performing a global nearest-neighbour vector search first, and then filtering results based on geographical metadata – the query will fail. If a user in Germany submits a photo of a European wasp, the top-50 visually nearest neighbours returned by an unconstrained vector index might consist entirely of visually indistinguishable American species. Applying a subsequent geographical filter then drops all 50 candidates, resulting in an empty retrieval set and complete context starvation.

In order to resolve this, the architecture adopts **Single-Stage Metadata-Filtered HNSW** implemented within LanceDB. LanceDB integrates Apache Arrow columnar data structures directly with vector indices, evaluating geographic and temporal boolean constraints natively *during* graph traversal rather than post-retrieval.

Let $\mathbf{x}_{\text{query}} = (\text{lat}_{\text{query}}, \text{lon}_{\text{query}})$ denote the GPS coordinates of the query, and $\tau_{\text{month}} \in \{1, \dots, 12\}$ denote the observation month. For each candidate node j encountered during the HNSW beam search, the traversal engine evaluates the deterministic predicate $\mathcal{P}(j)$:

$$\mathcal{P}(j) = [\mathcal{D}_{\text{geodesic}}(\mathbf{x}_{\text{query}}, \mathbf{x}_j) \leq R_{\max}] \wedge [\min(|\tau_{\text{month}} - \tau_j|, 12 - |\tau_{\text{month}} - \tau_j|) \leq \Delta_{\text{season}}],$$

where $\mathcal{D}_{\text{geodesic}}$ is the great-circle geographic distance computed via the Haversine equation, $\mathbf{x}_j$ represents the coordinates of exemplar node j, $R_{\max} = 500$ km is the maximum admissible geographic search radius, τ_j is the historical observation month of exemplar j, and $\Delta_{\text{season}} = 1$ enforces a 3-month seasonal window centred on the observation epoch across cyclic year boundaries. If node j violates $\mathcal{P}(j)$, its outgoing edges are pruned immediately, preventing the search from descending into biogeographically invalid graph clusters and guaranteeing 100% valid retrieval recall in under 5 ms.

5.4 Conformal-Driven Lazy Execution

Executing zero-shot entity linking, vector graph traversal, and generative multimodal reasoning for every incoming request imposes prohibitive latency and computational costs. In order to maintain an SLO of under 25 ms for high-volume inference, the Vision RAG pipeline utilises conformal-driven lazy execution governed by the 4-tier conformal routing contract:

1. **Decisive Classification ($|\hat{C}(X)| = 1$ with $\hat{\pi}_{\max}(X) \geq \tau_{\text{conf}}$):** The model exhibits high confidence and low epistemic ambiguity ($\hat{\pi}_{\max}(X) \geq \tau_{\text{conf}} = 0.60$). The endpoint dispatches the single predicted taxon immediately, fulfilling the strict sub-25 ms Service Level Objective (SLO) without activating retrieval overhead.
2. **Moderate Ambiguity ($1 < |\hat{C}(X)| \leq k_{\max}$, with $k_{\max} = 3$):** The conformal prediction set contains between two and three plausible sibling taxa. The serving engine executes a single-stage filtered HNSW traversal over LanceDB, constrained by the candidate class identifiers and spatio-temporal metadata, and performs Reciprocal Rank Fusion (RRF) over visual exemplar crops with less than 6 ms overhead to deliver a multi-candidate comparative differential to the operator.
3. **Severe Ambiguity or Low Top-1 Confidence ($|\hat{C}(X)| > k_{\max}$ or ($|\hat{C}(X)| = 1 \wedge \hat{\pi}_{\max}(X) < \tau_{\text{conf}}$)):** The prediction set exceeds the bound of actionable human differential review, or a singleton set fails the minimum confidence threshold ($\hat{\pi}_{\max} < 0.60$). The query is scheduled for asynchronous multimodal resolution via gRPC to the quantised visual language model (`taxon-vision-vlm`, based on Qwen2-VL-2B) and appended to the Active Learning triage queue (`triage_queue.lance`) on SeaweedFS for expert annotation.
4. **Epistemic Novelty / Complete Abstention ($|\hat{C}(X)| = 0$):** The observed sample violates the conformal coverage boundary, indicating an out-of-distribution observation, unmodelled developmental instar, or extreme sensor degradation. The system executes complete epistemic abstention, bypasses automated classification entirely, executes an unconstrained nearest-neighbour retrieval over the global exemplar space, and dispatches an immediate priority triage alert to human taxonomists.

5.5 Multi-Modal Re-Ranking via Reciprocal Rank Fusion

Nearest-neighbour vector search establishes raw visual affinity, but visual embeddings alone cannot capture fine morphological traits, such as minor wing venation patterns or subtle genital plate structures. In order to enhance diagnostic relevance, retrieved candidates are re-ranked

using Reciprocal Rank Fusion (RRF), integrating visual similarity with empirical biogeographical observation frequencies:

$$\text{Score}_{\text{RRF}}(i) = \frac{w_{\text{vis}}}{k + r_{\text{vis}}(i)} + \frac{w_{\text{geo}}}{k + r_{\text{geo}}(i)},$$

where $\text{Score}_{\text{RRF}}(i)$ is the consolidated ranking metric for candidate exemplar i, $w_{\text{vis}} = 0.65$ and $w_{\text{geo}} = 0.35$ denote scalar weighting factors for visual cosine similarity and biogeographical occurrence priors respectively, $k = 60$ is a rank-smoothing constant that prevents low-ranking outliers from dominating the fused score, $r_{\text{vis}}(i)$ is the ordinal rank of candidate i sorted descending by cosine similarity $z_{\text{child}} \cdot z_i$, and $r_{\text{geo}}(i)$ is the ordinal rank sorted descending by historical observation density $P(y_i \mid \text{Cell}_{\text{H3}}, \tau_{\text{month}})$ derived from pre-indexed GBIF occurrence frequencies.

This rank fusion guarantees that candidates appearing at the top of the context window are both visually representative of the query and frequently documented within that ecological spatio-temporal niche.

5.6 Multimodal Context Synthesis & VLM Diagnostics

When severe ambiguity ($|\hat{C}(X)| > k_{\max}$) or low top-1 separation confidence triggers Tier-3 escalation, or when an operator explicitly requests an automated differential diagnosis, the candidate exemplars resulting from Reciprocal Rank Fusion are assembled into a structured prompt schema along with the query observation. This payload is dispatched to a localised Vision-Language Model (such as a quantised Qwen2-VL-2B instance running on host P-cores or the RTX 5070 Ti).

In order to ensure machine readability and eliminate ungrounded generation or conversational drift, the VLM is constrained by grammar-based sampling to emit a strictly typed JSON payload:

```
{
  "conformal_context_size": 2,
  "candidates_evaluated": ["Bombus terrestris", "Bombus lucorum"],
  "primary_discriminator": "Pronotal collar colouration and width",
  "morphological_analysis": "The specimen exhibits a dark buff-yellow abdominal band and
  ↪  brownish-white tail hairs matching Bombus terrestris rather than the bright lemon-yellow band
  ↪  typical of Bombus lucorum.",
  "recommended_taxon": "Bombus terrestris",
  "epistemic_verdict": "HUMAN_TRIAGE_RECOMMENDED",
  "rationale": "High visual overlap under strong sunlight reflection; wing marginal cell venation
  ↪  partially occluded."
}
```

This structured response is rendered on the HTMX operator dashboard as a comparative differential diagnosis card. This allows human specialists to inspect aligned visual crops alongside identified discriminative keys.

5.7 Operationalising Taxonomic Mutations in the MMKG

The Vision RAG architecture functions as the ingestion engine for human-in-the-loop active learning workflows. When the Vision-Language Model assigns an epistemic verdict of `HUMAN_TRIAGE_RECOMMENDED` to an ambiguous query, the observation and its retrieved visual context are mirrored to the DagsHub Label Studio workspace. Once experts validate the taxonomy, the new annotations expand the training manifests and update the reference exemplar index.

However, beyond classifying new observations, the system must dynamically adapt to Linnaean taxonomic realignments. Biological taxonomies are fluid; the scientific community frequently splits a single species into multiple distinct taxa, or lumps distinct taxa into a single unified classification. Retraining a neural network or recalculating millions of vector embeddings every time a nomenclature committee publishes an update is computationally unviable.

The Multimodal Knowledge Graph resolves this by decoupling the immutable visual embedding vectors from the mutable relational metadata using the **DuckDB 2.0.0 LTS** analytical engine (codenamed "Cinnamon Teal") operating over LanceDB dataset tables via Apache Arrow zero-copy memory buffers:

- **Taxonomic Lumps (Merging):** When two distinct taxa (e.g., Species A and Species B) are combined into a single valid species (Species C), the pipeline does not re-embed any visual data. Instead, it executes an $\mathcal{O}(1)$ alias mapping operation. A background DuckDB 2.0.0 worker executes an in-place vectorized SQL update on the relational Lance manifest, pointing historical taxon keys for A and B to the unified key for C. The underlying dense vectors residing in the HNSW index remain untouched, updating the taxonomic knowledge base in milliseconds with zero GPU compute overhead.
- **Taxonomic Splits (Dividing):** When a recognised species is divided into two distinct taxa, historical visual crops associated with the parent taxon become scientifically ambiguous. The DuckDB reconciliation worker sets a boolean metadata flag `requires_reannotation = true` over those specific records in the Lance table. LanceDB's single-stage filtered HNSW traversal natively excludes nodes where `requires_reannotation = true`, preventing contaminated retrieval in production whilst routing the affected exemplars to Label Studio for expert sorting.

5.8 End-to-End Latency & Resource Allocation

Operating a decoupled serving architecture requires clear engineering resource bounds. To guarantee the overarching sub-25 ms Service Level Objective (SLO) on production GPU silicon (NVIDIA RTX 5070 Ti), the platform partitions the execution timeline into bounded budget allocations. Table 5.1 details these engineering budget ceilings across the fast hot-path and the lazy retrieval path.

Crucially, these figures represent allocated architectural SLO ceilings rather than static, universal hardware constants. Physical backbone inference latency varies substantially across execution providers: a domain transformer such as BioCLIP-2 or DINOv3 executes in under 9 ms when compiled with FP16 TensorRT or ONNX Runtime on modern desktop GPU Tensor Cores, but requires 20 to 50 ms or more when deployed on general-purpose CPUs without dedicated SIMD acceleration. Consequently, downstream operational deployments strictly profile candidate extractors dynamically using the Pareto benchmark suite rather than assuming static values from external literature. Furthermore, during active Jenkins GPU training windows, `taxon-vision-vlm` either scales to zero or offloads inference to the 16 host CPU P-cores and 128 GB system memory, guaranteeing that concurrent serving allocations never exceed the 4.8 GB VRAM reserve derived in Section 2.3.

Table 5.1: End-to-End Latency SLO Budget and Resource Allocation (GPU Serving Baseline)

Pipeline Segment	**Execution Engine**	**SLO (ms)**	**Memory (MB)**
Zero-Shot Cropping	YOLO-World (ORT INT8)	≤ 12.0	≤ 400 VRAM
VFM Feature Extraction	Vision Backbone (ORT INT8)	≤ 9.0	≤ 800 VRAM
Conformal Gate	PyTorch / C-Array	≤ 1.0	< 50 RAM
Filtered HNSW	LanceDB (Arrow + HNSW)	≤ 4.0	≤ 250 RAM
Multi-Modal RRF	In-Memory DuckDB / Polars	≤ 1.5	< 20 RAM
VLM Synthesis (Lazy)	Quantised 2B VLM	$\leq 2{,}500.0$	$\leq 2{,}500$ VRAM
Fast Hot-Path	**Crop + ORT Backbone**	$\mathbf{\leq 21.0}$	$\mathbf{\leq 1{,}200}$ **VRAM**
Lazy RAG Path	**Retrieval + VLM**	$\mathbf{\leq 2{,}600.0}$	$\mathbf{\leq 3{,}700}$ **VRAM**

Chapter 6

Epistemic Safety & Active Learning

Deploying deep neural networks within automated ecological monitoring workflows introduces a fundamental operational risk: discriminative classifiers optimised under cross-entropy objectives are systematically uncalibrated and prone to overconfident misclassifications. Because the softmax operator projects arbitrary logit vectors onto the probability simplex, the resulting class assignment is invariant to constant scalar shifts and sensitive only to relative logit differences. When confronted with motion-blurred field imagery, non-biological sensor artefacts, or uncatalogued taxa from outside the training distribution, standard vision architectures frequently concentrate probability mass onto a single erroneous category. In biodiversity informatics, such silent misclassifications corrupt longitudinal occurrence registries and distort downstream conservation models.

To establish verifiable production reliability, the TaxonVision-MLOps serving architecture decouples representation extraction from raw point predictions. The inference pipeline integrates three sequential mechanisms into a unified epistemic safety layer: pre-softmax Helmholtz free-energy filtering, deterministic spatial-temporal logit masking, and distribution-free Split Conformal Prediction. Together, these stages enforce bounded marginal error rates under arbitrary data distributions and programmatically route statistically ambiguous or novel observations into a human-in-the-loop active learning cycle.

6.1 Energy-Based Out-of-Distribution Detection

Unconstrained citizen-science data streams continuously introduce non-biological clutter, corrupted image payloads, and organisms belonging to unmodelled phyla. Constructing prediction sets or executing vector retrieval over such inputs wastes computational cycles and contaminates operational telemetry. Consequently, the initial gate of the safety layer intercepts and rejects out-of-distribution (OOD) inputs directly within the unnormalised logit space prior to probability normalisation.

Conventional anomaly detectors rely on the Maximum Softmax Probability (MSP), defined as $\max_j \hat{\pi}_j(x)$. However, because the unit-sum constraint $\sum_{j=1}^{K} \hat{\pi}_j(x) = 1$ forces probability mass to be conserved regardless of absolute activation magnitude, MSP exhibits high false-positive rates on out-of-distribution samples that trigger weak, near-uniform feature responses with a slight random perturbation. To circumvent this normalisation pathology, the platform evaluates the scalar Helmholtz free energy of the logit vector. Borrowed from statistical mechanics, the free energy $E(x; f)$ quantifies the negative log-partition function of the discriminative model, acting as a surrogate for the negative log-density of the input $x \in \mathcal{X}$ under the learned data manifold.

Let $f : \mathcal{X} \to \mathbb{R}^K$ denote the parameterised neural mapping from the normalised visual crop x to the unnormalised logit vector $f(x) = (f_1(x), \ldots, f_K(x))^\top$ across K operational taxa. Under the

Gibbs-Boltzmann distribution, the categorical likelihood of class $j \in \{1, \dots, K\}$ at temperature $T \in \mathbb{R}^+$ is coupled to the scalar free energy function $E(x; f)$ via:

$$E(x; f) = -T \cdot \log \sum_{j=1}^{K} \exp\left(\frac{f_j(x)}{T}\right) \tag{6.1}$$

where $T > 0$ denotes the temperature scaling parameter governing the smoothness of the log-sum-exp aggregation (fixed to $T = 1.0$ during standard inference). Inputs aligned with the learned training manifold elicit strong positive activations on compatible taxonomic classes, driving the partition sum upwards and yielding low (strongly negative) free energy values. Conversely, out-of-distribution anomalies fail to excite class-specific feature detectors, resulting in suppressed logit magnitudes and elevated free energy states.

6.1.1 Orthogonal Epistemic Defense via Predictive Shannon Entropy

Whilst the scalar Helmholtz free energy effectively detects non-biological noise, background clutter, and sensor dropouts that fail to elicit feature activations, energy thresholding alone exhibits an intrinsic vulnerability under fine-grained biological domain shifts. When confronted with an out-of-perimeter organism from an unmodelled biological class or phylum – for example, an insect larva or caterpillar submitted to a model trained exclusively on botanical taxa – the input features may align weakly with multiple disparate class weights simultaneously. The cumulative inner products across several classes can artificially inflate the partition sum $\sum_{j=1}^{K} \exp(f_j(x)/T)$, driving the free energy below τ_{OOD} despite the absence of any true botanical morphological homology.

To prevent such out-of-perimeter specimens from silently clearing the OOD gate, the platform couples Helmholtz energy with the predictive Shannon entropy $H(\hat{\pi}(x))$ of the softmax distribution. For the probability vector $\hat{\pi}(x) = (\hat{\pi}_1(x), \dots, \hat{\pi}_K(x))^\top \in \Delta^{K-1}$, the predictive entropy is evaluated in nats:

$$H(\hat{\pi}(x)) = -\sum_{j=1}^{K} \hat{\pi}_j(x) \log\left(\hat{\pi}_j(x) + \epsilon\right) \tag{6.2}$$

where $\epsilon = 10^{-12}$ provides numerical stability. The predictive entropy quantifies categorical dispersion: confident, decisive assignments yield $H \to 0$, whereas severe epistemic indecision over ambiguous or out-of-perimeter taxa drives H towards the uniform maximum $\log(K)$ ($H_{\max} = \log(10) \approx 2.3026$ nats for $K = 10$).

During real-time serving, the system evaluates both safety boundaries simultaneously. Downstream execution is governed by the composite binary gating operator $\Phi(x)$:

$$\Phi(x) = \begin{cases} 1 & \text{(In-Distribution: Admissible)}, \quad \text{if } E(x; f) \le \tau_{\text{OOD}} \ \wedge \ H(\hat{\pi}(x)) \le \tau_{\text{entropy}}, \\ 0 & \text{(Out-of-Distribution: Triage)}, \quad \text{if } E(x; f) > \tau_{\text{OOD}} \ \vee \ H(\hat{\pi}(x)) > \tau_{\text{entropy}}. \end{cases} \tag{6.3}$$

where the entropy threshold is set to $\tau_{\text{entropy}} = 1.60$ nats (in `config/default_config.yaml`) and the energy boundary τ_{OOD} is resolved dynamically from calibration artefacts (`config/conformal_config.yaml`, with `config/default_config.yaml` providing an offline baseline fallback; see Section 6.1.3). This dual-gate formulation guarantees that an input is admitted to spatial-temporal masking if and only if it exhibits both sufficient overall feature activation mass ($E \le \tau_{\text{OOD}}$) and decisive categorical concentration ($H \le \tau_{\text{entropy}}$).

6.1.2 Analytical Upper and Lower Energy Bounds

Unlike normalised probabilities, which are strictly confined to the unit interval $[0, 1]$, Helmholtz free energy operates on an unconstrained scale whose domain depends directly on the taxonomic cardinality K, the temperature T, and the L_2 weight-decay regularisation λ of the classification head. Adopting arbitrary static thresholds from external literature – such as $\tau_{\text{OOD}} = -12.5$, derived from unregularised 1,000-class ImageNet benchmarks – violates the mathematical domain of compact linear heads and induces complete rejection of valid observations. Establishing a

rigorous decision boundary therefore requires deriving the analytical upper and lower energy asymptotes of the classifier.

Upper Energy Bound (Maximum Entropy and the Noise Floor). Consider an uninformative input x_{noise} (such as background foliage, sensor blur, or an unmodelled phylum) that shares zero morphological features with the K catalogued taxa. Because the frozen vision backbone embeddings are unit-normalised ($\|z\|_2 = 1$) and the linear head weights are regularised towards the origin, the inner products vanish and the logit vector collapses towards zero-centred uniform indifference, $f_j(x_{\text{noise}}) \to 0$ for all $j \in \{1, \dots, K\}$. Substituting this zero-activation state into Equation 6.1 yields the analytical noise ceiling E_{noise}:

$$E_{\text{noise}}(K, T) = -T \cdot \log \left(\sum_{j=1}^{K} \exp(0) \right) = -T \cdot \log(K) \tag{6.4}$$

For the baseline operational catalogue of $K = 10$ taxa at unit temperature ($T = 1.0$), this upper bound evaluates to $E_{\text{noise}} = -\log(10) \approx -2.3026$. Physically, E_{noise} represents the state of maximum thermodynamic entropy: any observation yielding $E(x; f) \geq -2.3026$ possesses less total activation mass than uniform random noise and is deterministically classified as out-of-distribution clutter.

Lower Energy Bound (In-Distribution Activation Regime). To determine the lower energy floor E_{min} attainable by genuine biological specimens, let $f_{\max}(x) = \max_j f_j(x)$ denote the dominant class logit, and let $\hat{\pi}_{\max}(x) = \exp(f_{\max}(x)/T) / \sum_j \exp(f_j(x)/T)$ denote the corresponding maximum softmax probability. Factoring $\exp(f_{\max}(x)/T)$ out of the partition sum in Equation 6.1 yields the exact algebraic identity:

$$E(x; f) = -f_{\max}(x) + T \cdot \log \hat{\pi}_{\max}(x) \tag{6.5}$$

Because $\hat{\pi}_{\max}(x) \in [1/K, 1]$, the logarithmic correction term is strictly bounded within $\log \hat{\pi}_{\max}(x) \in [-\log K, 0]$. For a high-confidence in-distribution specimen where $\hat{\pi}_{\max}(x) \to 1$, the second term vanishes ($\log \hat{\pi}_{\max}(x) \to 0$), and the free energy converges asymptotically to the negative dominant logit, $E(x; f) \approx -f_{\max}(x)$. Under an L_2-regularised linear classification head with bounded weight norm $\|W\|_\infty$ operating on unit-hypersphere embeddings, the maximum achievable logit is bounded by the Cauchy-Schwarz inequality, $f_{\max}(x) \leq \|w_{\max}\|_2 \|z\|_2 + b_{\max}$. Empirically, across well-converged DINOv3 and BioCLIP-2 heads, prototypical specimens generate dominant logits in the interval $f_{\max}(x) \in [3.5, 4.8]$ with $\hat{\pi}_{\max}(x) \geq 0.85$, confining legitimate in-distribution observations to the energy band $E(x; f) \in [-4.80, -3.50]$. By contrast, an uncalibrated threshold of $\tau = -12.5$ would demand $f_{\max}(x) \geq 12.5$ ($\exp(f_{\max}) \approx 2.68 \times 10^5$), a regime mathematically unreachable under weight-decay regularisation.

6.1.3 Dynamic Energy Threshold Calibration and Configuration Invariants

Rather than relying on static analytical heuristics or hard-coded assumptions within general application settings, the decision boundary τ_{OOD} is evaluated dynamically as an empirical calibration step during Stage 6 of the pipeline (`src/taxon_vision/pipelines/calibrate.py`). Given the holdout calibration split $\mathcal{D}_{\text{cal}} = \{(x_i, y_i)\}_{i=1}^{n}$ (which remains strictly unexposed to linear head weight optimisation and Bayesian hyperparameter sweeps), the module passes the precomputed feature vectors through the compiled, INT8-quantised model graph (`model_int8.onnx`) to extract unnormalised logit vectors.

The empirical Helmholtz free-energy distribution $\mathcal{E}_{\text{cal}} = \{E(x_i; f)\}_{i=1}^{n}$ is computed in accordance with Equation 6.1. For a configured in-distribution True Positive Rate (TPR) of $1 - \beta = 0.95$, the calibrated threshold τ_{OOD} is calculated as:

$$\tau_{\text{OOD}} = \text{Quantile}\left(\mathcal{E}_{\text{cal}}; 1 - \beta\right), \quad \text{subject to } \tau_{\text{OOD}} < -T \cdot \ln(K). \tag{6.6}$$

To eliminate configuration split-brain anomalies between offline pipeline execution and production inference serving, Stage 6 serialises the computed parameters directly into `config/conformal_config.yaml`:

```yaml
conformal:
  alpha: 0.05
  quantile_threshold: 0.8182
  nominal_coverage: 0.95
ood:
  calibrated_energy_threshold: -2.4018
  entropy_threshold: 1.6000
```

At service initialisation, the runtime resolver (`_load_calibrated_ood_threshold()` in `src/taxon_vision/service/inference.py`) inspects `config/conformal_config.yaml`. If this file is present and validated against the internal Pydantic schema, the calibrated value overrides the static deployment baseline. The threshold specified in `config/default_config.yaml` is retained strictly as an offline fallback to ensure execution continuity during cold-start unit testing and initial container compilation prior to calibration data availability. During pipeline execution, the exact calibrated threshold boundary is logged directly into MLflow and annotated onto the validation energy histogram artefact (`visualizations/energy_distribution.png`).

Because the energy scale is intrinsically coupled to the logit geometry of the active model, τ_{OOD} is not a static constant and must undergo automated recalibration whenever one of three structural transitions occurs:

1. **Taxonomic Cardinality Expansion ($K \to K'$):** The baseline noise ceiling shifts logarithmically according to $\Delta E_{\text{noise}} = -T \cdot \ln(K'/K)$. Scaling the catalogue from $K = 10$ to $K' = 2{,}000$ taxa shifts the uniform noise floor from -2.30 to $-\ln(2000) \approx -7.60$, requiring an immediate recomputation of Equation 6.6.
2. **Head Retraining and Hyperparameter Updates:** Modifications to the Optuna-tuned weight-decay penalty λ, dropout rate p, or Class-Balanced Loss factor β_{CB} alter the L_2 norm of the linear head weights $\|W\|_2$, rescaling $f_{\max}(x)$ and shifting the empirical energy distribution $\mathcal{E}_{\text{val}}$.
3. **Covariate Shift and Drift-Triggered Recalibration:** When the real-time Prometheus drift monitor detects a statistically significant shift in penultimate pre-normalised feature activation norms ($W_1(\tilde{\mathcal{Z}}_{\text{base}}, \tilde{\mathcal{Z}}_{\text{prod}}) > \tau_{\text{drift}}$, evaluated prior to L_2 unit projection) or an anomalous spike in the OOD rejection rate, the CI/CD pipeline triggers a recalibration pass over the updated DVC validation manifest prior to promoting the next `Challenger` model.

6.2 Spatial-Temporal Constraint Masking

Visual classification in high-dimensional representation spaces is vulnerable to convergent evolution, wherein taxonomically distinct species inhabiting separate biogeographical realms evolve near-identical morphological traits. Whilst a purely visual classifier assigns comparable activation scores to such allopatric sibling taxa, physical geography and seasonal phenology impose strict deterministic constraints.

To enforce biogeographical validity without introducing secondary neural network latency, the serving engine applies a deterministic boolean mask directly to the post-OOD logit vector. Incoming citizen-science observations carry DarwinCore spatial-temporal metadata comprising WGS84 coordinates and an acquisition timestamp. During ingestion, the coordinates are projected onto a discrete Uber H3 hexagonal cell index $h \in \mathcal{H}$ (Resolution 4), and the timestamp is mapped to a calendar month $m \in \{1, \dots, 12\}$.

Let $\mathcal{A}(h, m) \subseteq \{1, \dots, K\}$ denote the admissible taxon subset derived from verified historical GBIF occurrence records within a geodesic radius $R_{\max}$ of cell h during the seasonal window $(m \pm 1)$

(mod 12). The spatially constrained logit vector $\tilde{f}(x) \in (\mathbb{R} \cup \{-\infty\})^K$ is obtained element-wise for each taxon $j \in \{1, \ldots, K\}$:

$$\tilde{f}_j(x) = \begin{cases} f_j(x), & \text{if } j \in \mathcal{A}(h, m) \\ -\infty, & \text{if } j \notin \mathcal{A}(h, m) \end{cases} \tag{6.7}$$

Projecting $\tilde{f}(x)$ onto the probability simplex via the softmax operator yields the biogeographically constrained categorical distribution $\hat{\pi}(x) \in \Delta^{K-1}$:

$$\hat{\pi}_j(x) = \frac{\exp(\tilde{f}_j(x))}{\sum_{k=1}^{K} \exp(\tilde{f}_k(x))} \tag{6.8}$$

Because $\exp(-\infty) = 0$, any taxon absent from the admissible regional and seasonal niche $\mathcal{A}(h, m)$ is assigned strictly zero probability mass ($\hat{\pi}_j(x) = 0$), redistributing the normalised density exclusively across ecologically plausible candidates prior to uncertainty quantification.

6.3 Distribution-Free Uncertainty via Split Conformal Prediction

Even after out-of-distribution filtering and biogeographical masking, point predictions derived from $\arg\max_j \hat{\pi}_j(x)$ lack formal statistical validity. To bound the operational error rate without relying on distributional assumptions or asymptotic approximations, the serving architecture wraps the constrained probability estimator in a Split Conformal Prediction layer [1]. This transformation converts raw point estimates into dynamic prediction sets $\hat{C}(X) \subseteq \{1, \ldots, K\}$ accompanied by a rigorous, finite-sample marginal coverage guarantee.

6.3.1 Calibration Protocol and Finite-Sample Quantile Estimation

Let $\mathcal{D}_{\text{cal}} = \{(x_i, y_i)\}_{i=1}^{n}$ denote an independent calibration split comprising n exchangeable observation pairs, held strictly disjoint from the training and hyperparameter validation partitions. For each calibration instance (x_i, y_i) with ground-truth Linnaean class index $y_i \in \{1, \ldots, K\}$, the system evaluates a scalar non-conformity score $s_i \in [0, 1]$ defined as the complement of the constrained softmax probability assigned to the true taxon:

$$s_i = s(x_i, y_i) = 1 - \hat{\pi}_{y_i}(x_i) \tag{6.9}$$

The score s_i quantifies the degree of epistemic disagreement between the model prediction and the ground truth: confident, correct classifications yield $s_i \to 0$, whereas severely miscalibrated or ambiguous predictions yield $s_i \to 1$.

For a user-prescribed miscoverage significance level $\alpha \in (0, 1)$ (fixed at $\alpha = 0.05$ to enforce a 95% nominal coverage target), the calibration engine sorts the non-conformity scores in non-decreasing order, $s_{(1)} \leq s_{(2)} \leq \cdots \leq s_{(n)}$, and computes the finite-sample corrected empirical quantile $\hat{q}_\alpha$:

$$\hat{q}_\alpha = \text{Quantile}\left(\{s_1, \ldots, s_n\}; \frac{\lceil (n+1)(1-\alpha) \rceil}{n} \right) = s_{(\lceil (n+1)(1-\alpha) \rceil)} \tag{6.10}$$

where the ceiling correction $\lceil (n+1)(1-\alpha) \rceil / n$ accounts for the discrete rank granularity of the finite sample size n, preventing systematic under-coverage on unseen test distributions.

6.3.2 Prediction Set Construction and Marginal Coverage Guarantee

During live inference on a newly submitted field observation X_{n+1} with unobserved true label $Y_{n+1} \in \{1, \ldots, K\}$, the conformal gate constructs the prediction set $\hat{C}(X_{n+1})$ by admitting all candidate taxa whose non-conformity scores do not exceed the calibrated cutoff $\hat{q}_\alpha$:

$$\hat{C}(X_{n+1}) = \{y \in \{1, \ldots, K\} : 1 - \hat{\pi}_y(X_{n+1}) \leq \hat{q}_\alpha\} = \{y \in \{1, \ldots, K\} : \hat{\pi}_y(X_{n+1}) \geq 1 - \hat{q}_\alpha\} \tag{6.11}$$

Under the condition that the calibration tuples $\{(X_i, Y_i)\}_{i=1}^{n}$ and the test tuple (X_{n+1}, Y_{n+1}) are exchangeable (requiring that the calibration split undergoes the identical post-OOD filtering and spatial-temporal masking transformations as test instances), the ranks of the non-conformity scores $\{s_1, \ldots, s_{n+1}\}$ are uniformly distributed over the permutations of $\{1, \ldots, n+1\}$. Consequently, the constructed set $\hat{C}(X_{n+1})$ satisfies the finite-sample marginal coverage theorem:

$$1 - \alpha \leq \mathbb{P}\left(Y_{n+1} \in \hat{C}(X_{n+1})\right) \leq 1 - \alpha + \frac{1}{n+1} \tag{6.12}$$

where the upper bound assumes continuous non-conformity score distributions without discrete point-mass ties (which may arise under discrete spatial masking or INT8 quantisation). Crucially, the distribution-free lower bound $1-\alpha$ holds across arbitrary visual feature representations and base classifier architectures provided exchangeability is preserved. The cardinality $|\hat{C}(X_{n+1})|$ adapts natively to local input difficulty: unambiguous, high-signal photographs collapse to singleton sets ($|\hat{C}| = 1$), whilst partially occluded or morphologically cryptic specimens expand into multi-element sets that faithfully bound the model's epistemic uncertainty.

6.4 Conformal Triage Routing & Active Learning Feedback

The prediction set cardinality $|\hat{C}(X_{n+1})|$ coupled with the maximum softmax probability $\hat{\pi}_{\max}(X_{n+1}) = \max_y \hat{\pi}_y(X_{n+1})$ serves as a deterministic control signal at the FastAPI serving boundary, partitioning incoming inference traffic into a four-tier operational execution contract:

1. **Decisive Fast-Path Resolution ($|\hat{C}(X_{n+1})| = 1$ with $\hat{\pi}_{\max} \geq \tau_{\text{conf}}$):** Exactly one taxon satisfies $\hat{\pi}_y(X_{n+1}) \geq 1 - \hat{q}_\alpha$, and the top-1 confidence simultaneously satisfies the operational confidence threshold $\hat{\pi}_{\max}(X_{n+1}) \geq \tau_{\text{conf}}$ (calibrated to $\tau_{\text{conf}} = 0.60$ in `config/conformal_config.yaml`). The classification is statistically conclusive at the $1 - \alpha$ significance level and is returned immediately by the INT8 ONNX runtime within the sub-25 ms hot-path SLO, completely bypassing secondary retrieval and operator triage.

2. **Moderate Ambiguity ($1 < |\hat{C}(X_{n+1})| \leq k_{\max}$, with $k_{\max} = 3$):** The conformal prediction set contains between two and three closely related sibling taxa. The serving engine lazy-loads Pipeline 2's LanceDB index, executes a single-stage filtered HNSW traversal constrained to the candidate classes and spatio-temporal metadata, and performs Reciprocal Rank Fusion (RRF) over exemplar crops with under 6 ms overhead to deliver a multi-candidate comparative differential to the operator.

3. **Severe Ambiguity or Low Top-1 Confidence ($|\hat{C}(X_{n+1})| > k_{\max}$ or ($|\hat{C}(X_{n+1})| = 1 \wedge \hat{\pi}_{\max} < \tau_{\text{conf}}$)):** Automated assignment is halted when the set cardinality exceeds the ambiguity ceiling $k_{\max}$ or when a singleton set fails the minimum confidence threshold ($\hat{\pi}_{\max} < 0.60$). The latter condition accounts for diffuse predictive states where a single class dominates only marginally (for example, $\hat{\pi}_{\max} = 0.482$ with $1 - \hat{q}_\alpha = 0.15$), which would otherwise evade set-size gating. The observation is scheduled for asynchronous multimodal resolution via gRPC to the quantised visual language model (`taxon-vision-vlm`, based on Qwen2-VL-2B), persisted locally, and routed to human expert verification.

4. **Epistemic Abstention and Novelty ($|\hat{C}(X_{n+1})| = 0$):** Every candidate taxon exhibits a softmax probability strictly below $1 - \hat{q}_\alpha$. An empty prediction set indicates a residual anomaly that cleared the dual OOD gate but is rejected by all modelled classes – characteristic of uncatalogued rare species, morphologically distinct instars, or regional invasive taxa. The system executes complete epistemic abstention, initiates an unconstrained nearest-neighbour search across the global exemplar graph, and flags the specimen for immediate priority taxonomic audit.

6.4.1 Active Learning Acquisition via Diverse Gradient Embeddings

Observations diverted via epistemic rejection ($\Phi(x) = 0$), set expansion ($|\hat{C}| > k_{\max}$), low top-1 confidence ($\hat{\pi}_{\max} < \tau_{\text{conf}}$), or complete abstention ($|\hat{C}| = 0$) are first written atomically to

local persistent storage via `persist_flagged_observation()` (`data/flagged_observations/`) to guarantee resilience against object store network partitions. They are subsequently ingested into `triage_queue.lance` on the in-cluster SeaweedFS object store, capturing the compressed WebP visual crop, the penultimate backbone feature representation $z(x)$, and the unnormalised logit vector $f(x)$. The backlog is synchronised with the DagsHub Label Studio annotation workspace. Because expert taxonomic review is strictly capacity-constrained, processing the backlog in first-in, first-out order is suboptimal: environmental bursts (such as seasonal insect emergences) frequently flood the queue with hundreds of visually redundant specimens. To maximise the marginal information gain per annotation hour, the triage engine orders pending candidates using Batch Active Learning by Diverse Gradient Embeddings (BADGE) [2, 4].

For each unlabelled candidate x in the triage pool $\mathcal{U}$ with penultimate backbone feature representation $z(x) \in \mathbb{R}^d$ ($d = 384$ for DINOv3-ViT-S/14; $d = 512$ for BioCLIP-2; $d = 768$ for ViT-B/16 baselines), let $\hat{y} = \arg\max_j \hat{\pi}_j(x)$ denote the model's current hallucinated pseudo-label. BADGE constructs a hypothetical gradient embedding $g_x \in \mathbb{R}^{d \times K}$ by evaluating the gradient of the cross-entropy loss $\mathcal{L}_{\text{CE}}$ with respect to the weights $W = (w_1, \ldots, w_K)^\top \in \mathbb{R}^{K \times d}$ of the final linear classification layer. For each class block $j \in \{1, \ldots, K\}$, the d-dimensional gradient component $g_x^{(j)}$ admits the closed-form expression:

$$g_x^{(j)} = \frac{\partial}{\partial w_j} \mathcal{L}_{\text{CE}} \left(f(x), \hat{y} \right) = \left(\hat{\pi}_j(x) - \mathbb{I}(\hat{y} = j) \right) \cdot z(x) \tag{6.13}$$

where $\mathbb{I}(\cdot)$ denotes the indicator function. Equation 6.13 simultaneously encodes both predictive uncertainty and visual representation geometry: when the classifier is uncertain, the scalar residual $(\hat{\pi}_j(x) - \mathbb{I}(\hat{y} = j))$ is large, amplifying the Euclidean norm $\|g_x\|_2$; meanwhile, the outer-product structure with $z(x)$ preserves the directional orientation of the specimen in feature space.

To assemble an annotation batch $\mathcal{B} \subset \mathcal{U}$ of size B, the acquisition module executes k-means++ seeding over the set of gradient embeddings $\{g_x : x \in \mathcal{U}\}$, iteratively sampling centroids with probability proportional to their squared Euclidean distance from previously selected points. This procedure guarantees that the resulting batch $\mathcal{B}$ spans high-magnitude gradients (high epistemic uncertainty) that are mutually orthogonal in parameter space (high morphological diversity).

Once validated by human taxonomists in Label Studio, confirmed observations are committed back into the DVC-versioned dataset manifests on a scheduled cadence. The newly labelled specimens augment the training split for underrepresented long-tail taxa and refresh the exchangeable calibration holdout $\mathcal{D}_{\text{cal}}$. On subsequent CI/CD training cycles, this closed feedback loop refines the linear decision boundaries and tightens the empirical quantile $\hat{q}_\alpha$, progressively reducing average conformal set cardinality $\mathbb{E}[|\hat{C}(X)|]$ whilst preserving the $1 - \alpha$ marginal coverage invariant.

Chapter 7

Observability, Testing, Deployment

Transitioning a deep learning system from a localised development environment into a production ecosystem fundamentally shifts the engineering challenge from model optimisation to system verification, runtime isolation, and continuous operational health monitoring. A machine learning service rarely fails through overt code exceptions; instead, it fails silently through data distribution shift, memory leaks under continuous streaming, Translation Lookaside Buffer thrashing, and uncalibrated epistemic decay.

In order to maintain production integrity, our deployment architecture rigorously couples containerised runtime encapsulation with Kubernetes (K8s) cluster orchestration, real-time statistical telemetry, and a multi-tiered verification framework. This chapter outlines the software engineering mechanisms governing containerisation, traffic routing, automated drift alerting, and offline testing suites.

7.1 Containerisation and Multi-Stage Runtimes

Standard Python environments managed via virtual environment tools are notoriously fragile in production. Differences in dynamically linked system libraries, CUDA driver mismatches on host kernels, and unversioned C-extension compilation dependencies frequently cause models to behave non-deterministically across infrastructure boundaries.

In order to guarantee bit-exact execution determinism, the platform packages the inference engine into an immutable container image using a multi-stage Docker build pipeline. This process isolates the build environment from the final execution runtime, substantially reducing the attack surface and final image volume.

7.1.1 Multi-Stage Image Compilation

The build pipeline is logically partitioned into two distinct stages:

1. **The Builder Stage:** Utilises a full Linux distribution equipped with C++ compilation toolchains, NVIDIA CUDA development toolkits, and the Pixi package manager. In this stage, Pixi resolves the exact dependency graph specified in `pyproject.toml` and locked in `pixi.lock`, compiling native C-extensions and pre-building optimised ONNX Runtime binaries.
2. **The Runtime Stage:** A minimal, distroless Debian-slim base image is initialised, containing strictly the shared Linux kernel interfaces and runtime CUDA execution libraries. The compiled virtual environment, static assets, and FastAPI service code are subsequently copied from the builder stage without carrying over compilers, debugging symbols, or package caches.

7.1.2 Kernel-Level Process Isolation

At runtime, execution security and resource determinism are enforced directly through robust Linux kernel primitives:

- **Namespaces:** The containerised process is systematically isolated across six discrete namespaces: Process ID (PID) isolation prevents inspection of host workloads, Network (NET) virtualisation seals internal socket bindings, Mount (MNT) controls filesystem scoping, and Inter-Process Communication (IPC) strictly prevents unauthorised shared memory access.
- **Control Groups (cgroups v2):** Hardware utilisation boundaries are enforced at the kernel boundary. The container runtime configures hard memory ceilings to entirely eliminate out-of-memory cascading failures and carefully pins CPU thread execution to prevent thread migration away from dedicated host performance cores.
- **Rootless Execution:** The container runs strictly under an unprivileged user namespace (`USER appuser`), mounting the root filesystem as read-only and routing temporary scratch buffers to an ephemeral, in-memory `tmpfs` volume.

7.2 K8s Cluster Orchestration and Workload Routing

Deploying a hybrid vision architecture – where routine queries resolve in sub-25ms hot paths whilst ambiguous queries trigger heavy generative multimodal retrieval – requires cluster orchestration capable of handling highly asymmetric latency profiles. Whilst the physical bare-metal server detailed in Chapter 2 serves as the primary deployment host executing a single-node Kubernetes cluster (via k3s), the system is described by declarative Kubernetes manifests that can be packaged as Helm charts in order to scale across managed multi-node cloud clusters when production traffic surges. These manifests are strictly configured to separate lightweight inference paths from resource-intensive Vision RAG pipelines.

7.2.1 Asymmetric Pod Architecture and Ingress Routing

Rather than packaging the fast-path ONNX classifier and the multi-gigabyte Vision-Language Model into a single monolithic pod, the cluster architecture deliberately splits the workload into two specialised microservices:

1. **The Primary Classifier Service (`taxon-vision-serving`):** A high-density, horizontally scaled deployment of lightweight FastAPI pods hosting the INT8 quantised ONNX backbone and the Split Conformal Prediction engine. These pods require minimal memory (allocated under 1.5 GB VRAM) and handle initial incoming traffic, resolving decisive single-candidate queries within the sub-25 ms SLO without invoking heavy secondary microservices.
2. **The Multimodal RAG Service (`taxon-vision-vlm`):** A specialised, resource-allocated deployment mounting dedicated GPU Tensor Cores to host the quantised 2-billion parameter Vision-Language Model and the in-memory HNSW vector index.

An ingress controller inspects incoming network payloads and load-balances observation traffic across the `taxon-vision-serving` pods. The serving engine executes the 4-tier conformal routing contract:

1. If the prediction set cardinality evaluates to $|\hat{C}(X)| = 1$ and top-1 confidence satisfies $\hat{\pi}_{\max}(X) \geq \tau_{\text{conf}}$ (calibrated to $\tau_{\text{conf}} = 0.60$), the primary pod returns the classification directly through the ingress to the client within the sub-25 ms SLO, bypassing secondary retrieval.
2. If the conformal layer indicates moderate ambiguity ($1 < |\hat{C}(X)| \leq k_{\max}$, with $k_{\max} = 3$), it triggers a single-stage filtered HNSW traversal over LanceDB and Reciprocal Rank Fusion over exemplar crops with under 6 ms latency overhead.

3. If the ambiguity is severe ($|\hat{C}(X)| > k_{\max}$) or a singleton set fails the minimum confidence threshold ($|\hat{C}(X)| = 1 \wedge \hat{\pi}_{\max}(X) < \tau_{\text{conf}}$), the primary service dispatches an asynchronous internal gRPC call to the `taxon-vision-vlm` service and persists the sample to `triage_queue.lance` on the in-cluster SeaweedFS object store for expert review.
4. If the prediction set is empty ($|\hat{C}(X)| = 0$), the system executes complete epistemic abstention, initiates an unconstrained nearest-neighbour search across global exemplars, and dispatches an immediate priority triage alert.

7.2.2 Self-Healing and Health Probing

In order to prevent cluster degradation caused by deadlocks or memory corruption, K8s enforces dual-tier health probes:

- **Liveness Probes:** The kubelet periodically polls the `/health` HTTP endpoint. If the FastAPI event loop becomes unresponsive or encounters an unrecoverable worker thread failure, the control plane immediately terminates the failing container and instantiates a clean replica.
- **Readiness Probes:** A separate `/health/ready` probe strictly verifies that internal model weights, ONNX Runtime inference sessions, and LanceDB index handles are loaded and validated in memory before the ingress controller ever routes live citizen-science observation traffic to the pod.

7.2.3 Horizontal Pod Autoscaling on Custom Telemetry

Standard K8s autoscaling typically relies on CPU or system memory thresholds, which unfortunately provide misleading operational signals for neural network workloads. A GPU-bound container can experience severe execution queuing whilst reporting surprisingly low CPU utilisation.

The cluster architecture supports Horizontal Pod Autoscaling (HPA) driven by custom Prometheus metrics via the K8s Custom Metrics API. The autoscaler continuously monitors two primary indicators:

1. **GPU VRAM Saturation and Compute Utilisation:** Sourced directly from NVIDIA Data Center GPU Manager (DCGM) metrics.
2. **Conformal Ambiguity Influx:** Sourced directly from the application's telemetry. If an environmental event (for example, sudden regional weather shifts causing blurred imagery) causes the proportion of multi-candidate conformal sets ($|C| > 1$) to spike, the HPA preemptively scales the `taxon-vision-vlm` pod pool to absorb the anticipated surge in multimodal retrieval passes.

On multi-node cloud clusters, HPA dynamically scales worker nodes. On the single-node bare-metal host with a 4-slice NVIDIA time-slicing profile, concurrency is bounded within static virtual GPU slice limits (1 training slice for Jenkins, 2 slices for `taxon-vision-serving`, and 1 slice for `taxon-vision-vlm`), buffering traffic spikes via asynchronous queuing and CPU worker offloading.

7.3 Telemetry, Metric Exporters, Concept Drift Monitoring

Unlike traditional software services that experience deterministic failure modes, machine learning systems are uniquely vulnerable to silent statistical degradation. A model endpoint can report 100% uptime and sub-millisecond HTTP response times whilst concurrently outputting meaningless ecological predictions due to seasonal distribution shifts, camera sensor variations, or geographic migrations. Thus, the platform embeds comprehensive statistical telemetry deeply into the serving runtime.

7.3.1 Prometheus Telemetry Instrumentation

The FastAPI application exposes a secure `/metrics` endpoint scraped at regular intervals by a Prometheus time-series collector. Monitored indicators comprehensively distinguish between active metrics currently wired in the telemetry runtime (`src/taxon_vision/monitoring/telemetry.py`) and Phase 3 planned counters:

- **Inference Latency Histograms (`taxon_inference_latency_seconds`):** High-resolution histograms tracking the duration of preprocessing, ONNX backbone execution, and conformal evaluation, effectively generating empirical p50, p95, and p99 latency distributions.
- **Inference Counter (`taxon_predictions_total`):** Tracks cumulative prediction volume partitioned by classification outcome status.
- **Human Referral Gauge (`taxon_human_referrals_active`):** Monitors the instantaneous volume of ambiguous or anomalous queries currently routed to human expert triage.
- **Phase 3 Telemetry Counters (Design Target):** Scheduled for Phase 3 deployment, dedicated collectors track the sliding-window conformal set cardinality distribution (`taxon_conformal_set_size`) and Helmholtz energy OOD rejection rates (`taxon_ood_rejections_total`).

7.3.2 Prometheus Cluster Realisation and TSDB Storage Mechanics

To eliminate external software-as-a-service dependencies and prevent public network telemetry egress, the time-series monitoring infrastructure is realised as an autonomous, in-cluster Prometheus daemon (`quay.io/prometheus/prometheus:v2.54.1`) deployed within the `taxon-vision` namespace of the local k3s cluster.

Service Discovery and Scraping Topology. The server operates on an explicit 15-second scraping interval, utilising the Kubernetes API discovery provider (`role: endpoints`). Pods provisioned under `deployment/taxon-vision-serving` declare declarative scraping metadata via annotations (`prometheus.io/scrape: "true"`, `prometheus.io/port: "8000"`). The endpoint scraper ingests telemetry exposed by the internal Prometheus client module (`src/taxon_vision/monitoring/telemetry.py`), collecting latency distributions, prediction throughput, and active referral states without intermediary proxy layers.

Native TSDB Storage Pipeline and Write Durability. The ingestion engine bypasses external relational databases, committing streaming metrics directly to Prometheus's native Time Series Database (TSDB). The runtime write pipeline is structured into three discrete storage stages:

1. **In-Memory Head Chunk:** Incoming telemetry samples are appended to an active memory-mapped head block, providing low-latency retrieval for real-time alerting queries.
2. **Write-Ahead Logging (WAL):** Concurrently with memory ingestion, raw sample blocks are sequentially appended to an on-disk WAL in 128 KiB segments. In the event of an ungraceful node restart or container eviction, the WAL is replayed on boot to reconstruct uncompressed head chunks, guaranteeing zero telemetry loss.
3. **Block Compaction and Long-Term Partitioning:** Every two hours, the active head block is cut into an immutable block directory comprising data chunks, an inverted index (`index`), and partition metadata (`meta.json`). A background compactor periodically merges adjacent blocks to minimise inode consumption and improve query efficiency.

Underlying Filesystem Invariants (BTRFS NOCOW). The TSDB persistent volume is bound to host storage at `/var/lib/taxon-vision/prometheus-data` across the host NVMe SSD. Because BTRFS utilises a Copy-on-Write (CoW) allocation policy by default, continuous appending to WAL segments and frequent two-hour block compactions inevitably induce severe physical block fragmentation and latency degradation over longitudinal runs. To enforce deterministic write

throughput, the persistent directory is provisioned on a dedicated BTRFS subvolume with the `NOCOW` attribute applied prior to initialisation:

```
mkdir -p /var/lib/taxon-vision/prometheus-data
chattr +C /var/lib/taxon-vision/prometheus-data
```

Local retention is capped at 30 days via `-storage.tsdb.retention.time=30d`, establishing a predictable storage footprint of less than 4.0 GB for the baseline cluster deployment.

7.3.3 Two-Tier Hybrid Feature Drift Architecture

For visual feature drift detection in the TaxonVision architecture, neither a single scalar projection nor a heavy multidimensional test alone suffices. Instead, the platform implements a **two-tier hybrid strategy** that balances real-time serving performance with offline statistical rigour, leveraging the distinct computational and statistical trade-offs of Maximum Mean Discrepancy (MMD) and the 1D Wasserstein-1 distance.

Dimension	Maximum Mean Discrepancy (MMD)	Wasserstein-1 Distance (Scalar Norms)
Mathematical Formulation	Evaluated in a Reproducing Kernel Hilbert Space (RKHS) using a Gaussian RBF kernel $k(z, z') = \exp\left(-\frac{\|z-z'\|^2}{2\sigma^2}\right)$ across the full D-dimensional feature embedding manifold ($\mathbb{R}^{384}$ / $\mathbb{R}^{512}$ / $\mathbb{R}^{768}$).	Evaluates the 1D Earth Mover's Distance by projecting D-dimensional pre-normalised representations onto scalar L_2 Euclidean norms $\|\tilde{z}\|_2 = \sqrt{\sum_{j=1}^{D} \tilde{z}_j^2}$, computing the linear-time lower bound $W_1(P_{\text{ref}}, P_{\text{curr}}) \geq \|\mathbb{E}[\|\tilde{z}_{\text{ref}}\|_2] - \mathbb{E}[\|\tilde{z}_{\text{curr}}\|_2]\|$.
Computational Complexity	**Quadratic/Polynomial** $\mathcal{O}((M+N)^2 \cdot D)$ due to pairwise kernel evaluations across M baseline and N production vectors.	**Linear** $\mathcal{O}(N \cdot D)$, executing in sub-milliseconds using vectorised NumPy array operations.
Sensitivity Profile	Captures subtle higher-order non-parametric distribution shifts, covariance mutations, and manifold rotations across all embedding dimensions.	Detects global vector magnitude, illumination, camera sensor variation, and energy shifts, but compresses directional rotations that leave average norm lengths unchanged. Crucially requires pre-normalised activations ($\tilde{z}$); unit-normalised vectors collapse identically to $\|z\|_2 \equiv 1.0$, rendering the probe blind to drift ($\|1.0 - 1.0\| = 0$).
Operational Fit	**Offline / CI/CD Quality Gates**: Computational cost makes it ideal for scheduled retraining audits and candidate model promotion gates.	**Real-Time Telemetry & Webhooks**: Extremely fast execution prevents event loop starvation on live FastAPI `/metrics` endpoints.

Table 7.1: Structural trade-offs and operational roles of MMD versus Wasserstein-1 scalar norm drift detection.

Pre-Normalised Representation Invariants and the Unit-Norm Collapse

A critical mathematical constraint arises when monitoring representation drift across deep vision foundation models. In visual retrieval and metric learning workflows, penultimate embedding

vectors $z \in \mathbb{R}^D$ are customarily projected onto the unit hypersphere via L_2 normalisation:

$$z = \frac{\tilde{z}}{\|\tilde{z}\|_2}, \quad \text{such that } \|z\|_2 \equiv 1.0, \quad \forall x \in \mathcal{X}, \tag{7.1}$$

where $\tilde{z} = f_{\text{VFM}}(x)$ denotes the raw, unconstrained output emitted by the penultimate layer of the vision backbone.

If the streaming drift detector naively computes the scalar Wasserstein-1 distance across these normalised representations, the metric experiences total mathematical collapse:

$$W_1(Z_{\text{base}}, Z_{\text{prod}}) \geq \left| \frac{1}{M} \sum_{i=1}^{M} \|z_i\|_2 - \frac{1}{N} \sum_{j=1}^{N} \|z'_j\|_2 \right| = |1.0 - 1.0| = 0.0. \tag{7.2}$$

Under unit normalisation, the scalar norm projection becomes an invariant constant, rendering the monitoring probe entirely blind to arbitrary covariate shift, severe sensor degradation, and out-of-distribution feature decay.

To preserve sensitivity to input distribution shift without introducing the computational overhead of high-dimensional matrix evaluations, the architecture enforces a strict tensor invariant: the fast-path ONNX graph and intermediate pipeline stages emit raw, pre-normalised activations $\tilde{z} \in \mathbb{R}^D$. The scalar monitoring probe computes the L_2 norm directly over these unconstrained vectors. By the Kantorovich-Rubinstein duality theorem, evaluating the difference in empirical first moments across pre-normalised vector lengths provides a non-trivial linear-time lower bound on the true multi-dimensional Wasserstein-1 metric:

$$W_1(\tilde{\mathcal{Z}}_{\text{base}}, \tilde{\mathcal{Z}}_{\text{prod}}) \geq \left| \frac{1}{M} \sum_{i=1}^{M} \|\tilde{z}_i\|_2 - \frac{1}{N} \sum_{j=1}^{N} \|\tilde{z}'_j\|_2 \right|. \tag{7.3}$$

Unit normalisation is deferred exclusively to downstream cosine distance computations and nearest-neighbour graph indexing within LanceDB.

Evidently AI Offline Validation, Data Drift Invariants, and Four-Gate Promotion

Whilst the linear-time Wasserstein-1 scalar projection (W_1) protects live serving throughput, complex visual domain shifts – such as subtle phenological variations, background vegetation changes, or localized sensor noise – may alter the geometry of the latent representation manifold without substantially shifting the empirical mean norm. To evaluate these higher-order distribution shifts without blocking production serving requests, the architecture incorporates an offline statistical quality gate powered by Evidently AI.

The drift auditing engine operates across two decoupled regimes: an automated CI/CD stage within the DVC pipeline (Stage 3.5, `data_drift`), and an hourly scheduled drift evaluation orchestrated by Airflow 3. The mathematical formulation and operational workflow operate as follows:

1. **Data Ingestion and Baseline Alignment:** The evaluator hydrates pristine reference calibration embeddings ($\mathcal{D}_{\text{cal}}$, tracked via DVC) alongside an accumulated production inference split ($\mathcal{D}_{\text{prod}}$, stored as unconstrained pre-normalised activations in PostgreSQL).
2. **Statistical Test Execution:** The suite evaluates non-parametric two-sample Kolmogorov-Smirnov (KS) tests across individual embedding dimensions at a significance threshold of $\alpha_{\text{drift}} = 0.05$. In parallel, it evaluates the two-sample Maximum Mean Discrepancy (MMD^2) over the full D-dimensional latent representation space using a Gaussian RBF kernel with bandwidth σ resolved via the median heuristic:

$$\text{MMD}^2(P, Q) = \frac{1}{M^2} \sum_{i=1}^{M} \sum_{j=1}^{M} k(z_i, z_j) - \frac{2}{MN} \sum_{i=1}^{M} \sum_{j=1}^{N} k(z_i, z'_j) + \frac{1}{N^2} \sum_{i=1}^{N} \sum_{j=1}^{N} k(z'_i, z'_j), \tag{7.4}$$

where $k(z, z') = \exp\left(-\frac{\|z-z'\|^2}{2\sigma^2}\right)$. To calculate an exact non-parametric empirical p-value without asymptotic normal approximations, the engine executes $B = 1{,}000$ Monte Carlo permutations across pooled sample matrices:

$$p = \frac{1 + \sum_{b=1}^{B} \mathbb{I}\left(\mathrm{MMD}_b^2 \geq \mathrm{MMD}_{\mathrm{obs}}^2\right)}{B + 1}. \tag{7.5}$$

3. **Dual Artefact Emission:** The execution run generates two distinct artefacts:
   - `reports/drift_report.html`: An interactive, visual dashboard rendering drift distributions, categorical probability shifts, and per-feature empirical cumulative distribution functions (eCDFs), archived directly to DagsHub as a permanent build artefact.
   - `reports/drift_metrics.json`: A structured, machine-readable summary recording global drift share (the proportion of drifted feature dimensions), the scalar MMD distance, and the permutation p-value.
4. **Stage 7 Four-Gate Model Promotion Verdict:** Historical pipelines promoted models at Stage 4 based solely on validation Macro PR-AUC. This represents a dangerous anti-pattern: Stage 4 cannot evaluate INT8 post-training quantisation collapse, holdout conformal coverage, empirical inference latency, or exchangeability against live traffic. Consequently, promotion authority is transferred strictly to Stage 7 (`src/taxon_vision/pipelines/evaluate.py`). A candidate model earns the `Challenger` or `Production` alias if and only if it simultaneously satisfies all four promotion gates:
   - **Gate 1 (Accuracy Superiority):** Validation Macro PR-AUC must improve upon the active production model (ΔPR-AUC ≥ 0).
   - **Gate 2 (Quantisation Fidelity):** Top-1 accuracy degradation induced by INT8 Post-Training Quantisation must remain $\leq 3.0\%$ relative to the unquantised FP32 PyTorch baseline.
   - **Gate 3 (Empirical Conformal Coverage):** Empirical marginal coverage on the holdout test partition must satisfy the lower statistical bound $\geq (1 - \alpha) - 5\%$ (e.g. $\geq 90\%$ at nominal $\alpha = 0.05$).
   - **Gate 4 (Latency Ceiling & Live Exchangeability):** The p95 CPU inference latency must remain strictly below the Service Level Objective ceiling (≤ 120 ms), and the two-sample RBF MMD^2 permutation test against live PostgreSQL prediction telemetry must confirm covariate exchangeability ($p \geq 0.01$).

7.3.4 Dual-Path Alertmanager Routing and Automated Feedback Loops

Prometheus Alertmanager and Grafana evaluate time-series rules across live cluster metrics, enforcing a bifurcated routing topology that separates automated infrastructure remediation from human expert escalation:

1. **Path A: Automated Retraining Trigger (Machine Loop):** Triggered when empirical feature drift breaches the streaming tolerance ($W_1(\tilde{\mathcal{Z}}_{\mathrm{base}}, \tilde{\mathcal{Z}}_{\mathrm{prod}}) > \tau_{\mathrm{drift}}$) or when the multivariate MMD^2 permutation test detects significant manifold shift ($p < 0.01$). Grafana Alertmanager evaluates the PromQL rule `taxon_drift_detected == 1` and fires an authenticated HTTP POST payload to the FastAPI webhook relay (`/api/v1/retrain/webhook`) authenticated via `TAXON_RETRAIN_WEBHOOK_SECRET`. The relay verifies the secret, enforces an anti-flapping cooldown period, acquires a short-lived JSON Web Token (JWT) from Airflow 3, and triggers the `model_retraining` DAG via the Airflow REST API. Airflow merges verified Label Studio exports into `data/manifests/dataset_manifest.parquet`, triggers the Jenkins GPU pipeline (`job/taxon-vision-mlops/buildWithParameters?RETRAIN_MODEL=true`), monitors execution to completion, and evaluates the Stage 7 Four-Gate Promotion Verdict.

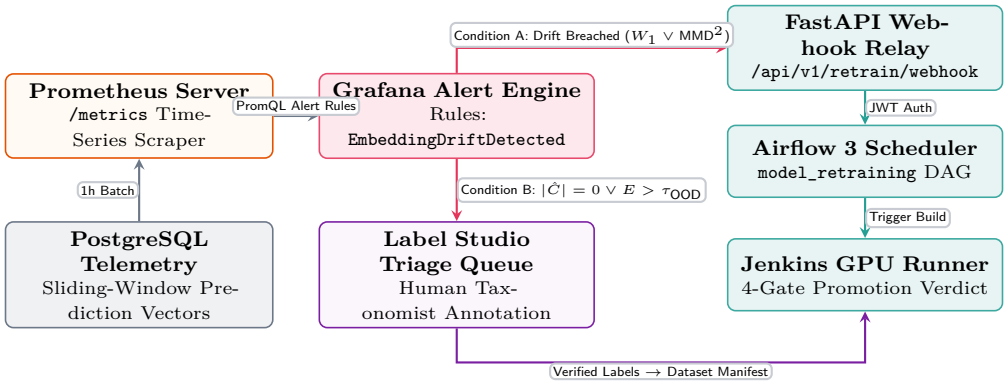


Figure 7.1: Dual-path monitoring and closed-loop continuous retraining architecture. Prometheus scrapes streaming metrics while PostgreSQL records sliding-window prediction embeddings. Under Condition A (feature or manifold drift), Grafana dispatches an authenticated webhook to the FastAPI relay, prompting the Airflow 3 scheduler to trigger an ephemeral GPU retraining job in Jenkins governed by the Stage 7 Four-Gate Promotion Verdict. Under Condition B (epistemic novelty or empty conformal sets), anomalous samples route to the Label Studio triage queue for expert re-annotation.

2. **Path B: Epistemic Abstention Alert (Human Loop):** Triggered when the rate of empty conformal prediction sets ($|\hat{C}(X)| = 0$) or severe out-of-distribution energy rejections ($E(x) > \tau_{\text{OOD}}$) exceeds 5% of total request volume. Because an empty prediction set indicates an organism that cannot be assigned to any catalogued class at the 95% confidence level, automated retraining cannot resolve the underlying novelty. Alertmanager formats a diagnostic alert detailing client coordinates, image hashes, and top logit distributions, routing it via Discord webhook and administrative email to the operational taxonomist team, whilst samples are staged in the Label Studio triage queue for expert re-sorting.

7.4 Multi-Tier Verification and Testing Strategy

A continuous integration pipeline for machine learning must validate considerably more than simple syntax correctness. The TaxonVision-MLOps testing framework strictly enforces a four-tiered verification suite executing on every commit.

7.4.1 Tier 1: Upstream API Mocking

During automated CI builds, executing real network requests against the live iNaturalist API violates rate-limiting policies and inevitably introduces non-deterministic network latency.

The test harness uses the `responses` library to intercept outgoing HTTP calls at the transport layer. The test runner injects mock JSON responses simulating diverse API states: valid DarwinCore metadata payloads, rate-limited responses (HTTP 429), server timeout conditions (HTTP 504), and malformed taxonomic trees. This rigorously guarantees that ingestion, licence filtering, and coordinate parsing logic are safely validated in under 5 seconds without any network dependencies.

7.4.2 Tier 2: Object Storage and Pipeline Mocking

In order to test Data Version Control (DVC) pipelines and S3 streaming operations without incurring AWS billing or requiring live DagsHub credentials, the integration suite employs the `moto` testing library.

The test setup instantiates an in-memory, ephemeral AWS S3 service directly within the local pytest process. The DVC client is pointed towards this virtual endpoint, allowing the test runner

to strictly verify cryptographic hash tracking, multi-part binary streaming, and dataset version switching entirely in local memory.

7.4.3 Tier 3: Mathematical Invariant Testing

Standard software testing relies heavily on binary assertions (for example, verifying an output equals a known constant). Machine learning models, however, are inherently probabilistic. The pipeline introduces robust mathematical invariant tests that evaluate statistical coverage guarantees well before model serialisation.

The test runner loads the newly trained ONNX candidate model and evaluates it against an unseen holdout test split $\mathcal{D}_{\text{test}} = \{(x_i, y_i)\}_{i=1}^{n_{\text{test}}}$. The test formally asserts that the empirical coverage mathematically satisfies the statistical bound:

$$\text{Empirical Coverage} = \frac{1}{n_{\text{test}}} \sum_{i=1}^{n_{\text{test}}} \mathbb{I}\left(y_i \in \hat{C}(x_i)\right) \geq 1 - \alpha,$$

where n_{test} denotes the cardinality of the holdout split, (x_i, y_i) is the i-th test observation with ground-truth Linnaean label, $\hat{C}(x_i)$ is the conformal prediction set calibrated at nominal significance level $\alpha = 0.05$, and $\mathbb{I}(\cdot)$ is the indicator function. If the empirical coverage drops below 0.95, the test fails with a fatal error, mathematically preventing an under-covered, uncalibrated model from ever being packaged into a release artefact.

7.4.4 Tier 4: Statistical Metric Selection and Anomaly Verification

In traditional computer vision benchmarking, the Area Under the Receiver Operating Characteristic curve (AUROC) is widely utilised to quantify model discrimination. In biological species monitoring and ecological anomaly detection, however, **AUROC is fundamentally flawed and actively deceptive**.

The Failure of AUROC under Ecological Class Imbalance

Biological datasets are heavily governed by extreme power-law distributions: common species possess tens of thousands of observations, whilst rare long-tail species and genuine out-of-distribution anomalies account for a minuscule fraction of the data.

The AUROC metric mathematically evaluates the trade-off between the True Positive Rate (TPR) and the False Positive Rate (FPR):

$$TPR = \frac{TP}{TP + FN}, \quad FPR = \frac{FP}{FP + TN}$$

where TP, FP, TN, and FN denote the counts of true positives, false positives, true negatives, and false negatives, respectively.

Because the negative class (normal, common organisms or background pixels) vastly outnumbers the anomalous class, the denominator of the False Positive Rate ($FP + TN$) is aggressively dominated by TN. Consequently, an algorithm can easily generate thousands of false positive alarms (FP), yet the calculated FPR remains near zero due to the massive TN divisor. A model exhibiting a high AUROC score of 0.98 can simultaneously suffer an acute precision collapse in production, burdening domain experts with false alerts.

Mandating PR-AUC and AUPIMO

In order to permanently prevent misleading verification gates, the testing suite explicitly rejects AUROC in favour of Precision-Recall Area Under the Curve (PR-AUC) and Area Under the Per-Image Overlap (AUPIMO).

For classification and global Out-of-Distribution anomaly detection, the pipeline robustly evaluates PR-AUC:

$$\text{PR-AUC} = \int_0^1 \text{Precision}(R)\, dR,$$

where $\text{Precision} = \frac{TP}{TP+FP}$ and $\text{Recall}(R) = \frac{TP}{TP+FN}$. Because PR-AUC mathematically excludes True Negatives (TN) from its formulation entirely, it directly reflects the true proportion of genuine biological anomalies amongst all flagged alerts, thus providing an honest measure of real-world efficacy.

For zero-shot visual cropping, bounding-box isolation, and spatial anomaly segmentation, the test harness enforces the Area Under the Per-Image Overlap (AUPIMO) metric. Unlike global pixel-level metrics that allow large contiguous background areas to distort evaluation scores, AUPIMO computes the intersection-over-union metric independently across each individual image's bounding box over varying threshold levels:

$$\text{AUPIMO} = \frac{1}{N_{\text{images}}} \sum_{k=1}^{N_{\text{images}}} \int_0^{\tau_{\max}} \Omega_k(\tau)\, d\tau,$$

where N_{images} is the total number of test photographs evaluated in the spatial verification suite, τ denotes the detection threshold integrated up to operational bound $\tau_{\max}$, and $\Omega_k(\tau)$ represents the per-image overlap rate for photograph k at threshold τ, evaluated as the ratio of correctly identified anomaly pixels within the ground-truth organism mask relative to the total organism area.

By evaluating overlap per image rather than across the pooled pixel space, AUPIMO ensures that the zero-shot visual entity linking module isolates small organisms (for example, small insects or solitary flowers) just as reliably as large organisms (such as large mammals or mature trees). This mathematical guarantee ensures that upstream cropping logic introduces zero systematic taxonomic bias into downstream retrieval.

7.4.5 Tier 5: Codebase Health, Test Coverage, and Cognitive Complexity

Beyond functional and statistical correctness, production maintainability mandates strict architectural hygiene across the codebase. The TaxonVision-MLOps verification framework couples comprehensive automated testing with formal complexity, AST code invariants, and cryptographic attestation:

1. **Comprehensive Automated Test Harness:** The regression test suite comprises **203 automated pytest tests** maintaining an empirical statement coverage of 89% across the entire `src/taxon_vision` package (with critical data, inference, and uncertainty modules exceeding 95%).
2. **Cognitive Complexity Invariants (≤ 15):** To prevent monolithic control flow anti-patterns, the CI quality gate executes automated cognitive complexity analysis via `complexipy`. Every function across both application code (`src/`) and governance scripts (`scripts/`) is constrained to a cognitive complexity ceiling of ≤ 15. Following modular decomposition of the OKF validation and visualisation engines, 100% of functions satisfy this ceiling with zero snapshot exemptions.
3. **AST Code Invariants Engine:** To programmatically prevent runtime regressions and anti-patterns, an automated Abstract Syntax Tree engine (`scripts/governance/verify_code_invariants.py`) enforces non-negotiable architectural mandates: (a) zero dynamic reflection (`hasattr`/`getattr`), (b) positional parameter arity ≤ 4 encapsulated in typed Pydantic models, (c) zero blocking synchronous I/O within asynchronous FastAPI routes, and (d) zero bypass comments (`# noqa`, `# type: ignore`, `# complexipy: ignore`). Legacy exceptions are strictly ratcheted downwards via `code_invariants_baseline.json`.

4. **Unified 8-Gate Attestation Seal:** Prior to cryptographic git commit signing (`git commit -S`), the platform executes a deterministic attestation runner (`.agents/verifiers/attest_seal.py`) verifying all eight pre-commit gates in sequence: Ruff linting and formatting, strict MyPy type checking across 70 source files, Complexipy complexity invariants, OKF v0.2 knowledge graph validation (`scripts/governance/validate_okf.py`), Markdown syntax and British English editorial idiom verification (`scripts/governance/verify_markdown_rules.py`), bare-metal hardware invariant specifications (`scripts/governance/verify_hardware_invariants.py`), open-licensing compliance (`taxon_vision.pipelines.audit_license`), and fast-path targeted test suites. Successful executions emit an immutable computation receipt to `.agents/receipts/`.

7.4.6 Tier 6: Persistent Bayesian Telemetry and Deployment Smoke Testing

Iterative hyperparameter tuning introduces volatile state risks when trials execute inside ephemeral container agents. To eliminate unpersisted trial histories, the architecture establishes a dedicated Bayesian telemetry plane:

1. **Optuna PostgreSQL Storage with SQLite Fallback:** The hyperparameter runner (`src/taxon_vision/models/training/runner.py`) binds to the consolidated PostgreSQL cluster using dialect `postgresql+psycopg2://optuna:...@postgres:5432/optuna`, matching the bundled driver in the official `optuna-dashboard` image. SQLite WAL mode (`sqlite:///data/optuna/optuna_studies.db`) is retained strictly as an automatic fallback for offline local workstation development.

2. **Cluster Optuna Dashboard:** Deployed in Kubernetes via `deploy/k8s/mlops-storage-and-optuna.yaml` using the official image (`ghcr.io/optuna/optuna-dashboard:latest`), the interactive dashboard is exposed to the local network via NodePort `30081`. It renders live optimisation history plots, parameter importances, and multi-objective Pareto frontiers across classification heads and backbone extractors.

3. **Post-Rollout Smoke Testing:** Following every zero-downtime rolling restart, the automated verification script (`scripts/ci/verify_deployment.sh`) probes the live cluster: (a) verifying rollout status of `deployment/taxon-vision-serving`, (b) testing `/health` and `/health/ready` probes, (c) dispatching a multipart synthetic image prediction to `/api/v1/predict`, and (d) asserting that Prometheus metrics counter `taxon_predictions_total` increments deterministically.

Chapter 8

Hardware Optimisation and Memory Mathematics

Operating a high-throughput machine learning service under strict latency service-level objectives (SLOs) requires moving decisively beyond theoretical model performance and addressing the physical limitations of silicon. This chapter explicitly defines the mathematical transformations utilised to compress neural network memory footprints, the formal theorems rigorously governing epistemic safety thresholds, and the memory complexity equations required to safely bound vector database consumption.

8.1 Static and Dynamic Quantisation Architectures

Deep neural networks are fundamentally composed of dense, high-dimensional weight matrices. By default, standard training frameworks construct these matrices using 32-bit floating-point (FP32) numerical representations. Whilst FP32 provides extreme gradient precision during the backpropagation phase of training, it is highly inefficient during production inference. FP32 tensors consume vast amounts of GPU VRAM, and executing matrix multiplications at this precision severely bottlenecks memory bandwidth, thereby inflating prediction latency.

In order to optimise the deployment architecture, the PyTorch classification head and feature extractor are compiled into the Open Neural Network Exchange (ONNX) format with dual output nodes ($f(x) \in \mathbb{R}^K$, $z(x) \in \mathbb{R}^D$), cleanly decoupling the computational graph from Python's interpreter overhead. Subsequently, the ONNX Runtime applies Integer-8 (INT8) Post-Training Quantisation (PTQ). This mathematical operation maps the continuous floating-point weights and activations onto a discrete 8-bit integer grid.

The linear quantisation mapping function is formally defined as:

$$q = \text{clamp}\left(\text{round}\left(\frac{x}{S}\right) + Z,\, q_{\min},\, q_{\max}\right) \tag{8.1}$$

where $q \in [q_{\min}, q_{\max}]$ denotes the 8-bit quantised integer (typically $[-128, 127]$ for signed INT8 or $[0, 255]$ for unsigned uint8), x represents the continuous 32-bit floating-point activation or weight value, $S \in \mathbb{R}^+$ is the positive real scale factor defining the step size between discrete quantised bins, and $Z \in \mathbb{Z}$ is the integer zero-point offset that aligns real-world mathematical zero with the discrete grid to ensure zero-padding operations remain exact.

The platform implements two distinct quantisation paradigms depending on operational context:

1. **Static INT8 QDQ Quantisation with Empirical Calibration:** Static Post-Training Quantisation inserts explicit QuantizeLinear/DequantizeLinear (QDQ) operator pairs surrounding convolutional and linear layers. The dynamic range of intermediate activation tensors

is calibrated empirically using a dedicated calibration dataset reader (`TaxonCalibrationDataReader`) streaming genuine biological image batches. For symmetric signed quantisation, the scale factor S is computed using MinMax calibration:

$$S = \frac{\max\left(|x_{\min}|,\, |x_{\max}|\right)}{127}, \quad Z = 0 \tag{8.2}$$

Static calibration eliminates runtime scaling overhead entirely, enabling fused integer matrix multiplications on hardware accelerators.

2. **Dynamic INT8 Quantisation with Logit Node Preservation:** In resource-constrained CPU environments lacking calibration data, dynamic quantisation quantises weights statically to INT8 whilst calculating activation scale factors dynamically at inference time. Crucially, as implemented in `quantizer.py`, the terminal linear projection layers directly emitting classification logits are excluded from aggressive quantisation. Preserving FP32 precision on logit output nodes prevents rounding distortions that would otherwise perturb the delicate unnormalised activation margins required by the Helmholtz energy detector and Split Conformal Prediction.

By mapping 32-bit floats to 8-bit integers, the model's theoretical tensor parameter footprint is deterministically reduced by 75% (an exact $4:1$ bit-width ratio; for example, reducing an illustrative 400 MB model to approximately 100 MB). Because INT8 operands are significantly smaller, modern hardware can efficiently pack multiple operations into single SIMD (Single Instruction, Multiple Data) registers, strongly leveraging AVX-512 Vector Neural Network Instructions (VNNI) on CPUs or Tensor Cores on GPUs. In published deployment benchmarks across standard vision backbones, this quantisation pattern yields an empirical $2\times$ to $4\times$ latency acceleration with typically less than a 1% degradation in top-1 accuracy (subject to validation on target biological splits).

8.2 Marginal Coverage Guarantees in Conformal Prediction

Standard classification models merely output uncalibrated probability distributions. In ecological monitoring, uncalibrated deep neural networks exhibit overconfidence, frequently assigning extreme softmax probabilities to out-of-distribution or corrupted observations. Relying on raw softmax scores introduces unquantified epistemic risk into automated observation catalogues.

To eliminate this failure mode, the architecture wraps the base classifier in a Split Conformal Prediction layer. Rather than forcing the model to emit a single point prediction, conformal prediction evaluates input ambiguity and generates a dynamic *set* of candidate taxa. When the input observation exhibits unambiguous morphological features, the prediction set collapses to a singleton. If the image is occluded or ambiguous, the set expands into a multi-candidate differential to preserve rigorous error guarantees.

Formally, this epistemic safety is guaranteed by the marginal coverage theorem:

$$\mathbb{P}\left(Y_{n+1} \in \hat{C}(X_{n+1})\right) \geq 1 - \alpha,$$

where $\mathbb{P}$ denotes the probability measure taken over the exchangeable joint distribution of images and taxonomic labels, X_{n+1} represents the unseen test observation, $Y_{n+1} \in \{1, \ldots, K\}$ is the unobserved true ground-truth taxonomic label, $\hat{C}(X_{n+1})$ is the dynamic prediction set systematically generated by the conformal algorithm containing zero, one, or multiple candidate taxa, and $\alpha \in (0, 1)$ is the nominal significance error tolerance (fixed at $\alpha = 0.05$ to enforce a 95% marginal coverage guarantee).

This theorem provides a remarkably robust distribution-free guarantee: at least $1 - \alpha$ of future predictions are mathematically guaranteed to contain the true species within the returned prediction set, holding under exchangeability between calibration and test observations without parametric assumptions on the underlying feature distributions.

8.3 Memory Complexity of Hierarchical Vector Graphs

Retrieval-augmented generation systems frequently bottleneck on disk I/O when attempting to query massive vector databases stored on solid-state drives. In order to guarantee sub-millisecond retrieval latency during the Single-Stage Metadata-Filtered HNSW traversal, the entire Reference Exemplar Catalogue must clearly fit completely within the host system's Random Access Memory (RAM).

In order to achieve this, the architecture establishes an illustrative engineering sizing model. Assuming an operational scope scaling to a design baseline of 10,000 distinct taxa, with 50 curated environmental crops per taxon, the vector index reaches a safe upper bound of 500,000 nodes.

The theoretical memory lower bound $\mathcal{M}_{\text{index}}$ required to securely store the raw embedding vectors in memory is mathematically calculated as:

$$\mathcal{M}_{\text{index}} = N_{\text{nodes}} \times D_{\text{dim}} \times B_{\text{bytes}},$$

where $\mathcal{M}_{\text{index}}$ is the theoretical memory footprint required to store the raw embedding vectors in RAM, $N_{\text{nodes}} = 500{,}000$ is the upper bound of curated biological exemplars, D_{dim} is the feature dimensionality of the chosen Vision Foundation Model, and $B_{\text{bytes}} = 4$ is the byte-size of 32-bit floating-point (FP32) numerical representations.

Whilst operational models leverage DINOv3 ($D_{\text{dim}} = 384$) or BioCLIP-2 ($D_{\text{dim}} = 512$), adopting $D_{\text{dim}} = 768$ (corresponding to large ViT architectures) establishes a conservative upper bound:

$$\mathcal{M}_{\text{index}} = 500{,}000 \times 768 \times 4 \text{ bytes} = 1{,}536{,}000{,}000 \text{ bytes} \approx 1.536 \text{ GB}.$$

Under this conservative assumption, the entire uncompressed vector payload occupies at most 1.536 GB (collapsing to 0.768 GB under DINOv3 or 1.024 GB under BioCLIP-2). Even when accounting for graph navigational links in HNSW, the complete index consumes well under 3.0 GB of host memory, allowing the entire retrieval graph to reside in fast DDR5 memory alongside serving workers and preserving the sub-25 ms inference SLO.

Chapter 9

Conclusion and Operational Synthesis

The TaxonVision-MLOps architecture demonstrates that transitioning from experimental, notebook-bound biological modelling to a globally scalable, Level 4 MLOps platform is not merely a matter of provisioning more cloud compute. It requires a fundamental paradigm shift towards deterministic engineering, epistemic safety, and decoupled operational lifecycles.

By completely abandoning monolithic web servers and fragile Python environments in favour of a strictly containerised, dual-pipeline Kubernetes topology, the system mathematically isolates the slow, weight-centric training cycles of the classification engine from the highly volatile, data-centric taxonomic mutations of the Multimodal Knowledge Graph (MMKG). The integration of zero-copy Apache Arrow streaming and DVC cryptographic Parquet manifests successfully circumvents the unmanageable 50-terabyte storage trap, allowing the platform to ingest massive citizen-science streams on highly constrained, cost-effective bare-metal infrastructure.

Crucially, the platform refuses to trade scientific reliability for inference speed. The implementation of Energy-Based Out-of-Distribution rejection and Split Conformal Prediction guarantees that no identification reaches production without a formal, distribution-free mathematical error bound. By enforcing these invariants within a deterministic CI/CD verification gate and routing statistically ambiguous queries through a Reciprocal Rank Fused Vision RAG pipeline, the architecture closes the active learning loop.

TaxonVision-MLOps establishes an epistemically secure intelligence platform capable of sustaining operational biological monitoring at global scale.

9.1 Phased Delivery Roadmap and Implementation Status

The architecture described in this report is delivered in three phases that end in a final defence. The milestones are fixed as follows: Phase 1 (foundations: reproducible environment, data ingestion, baseline model and inference interface) on 12 October 2026, Phase 2 (containerised services, experiment tracking and dataset versioning, scheduled retraining) on 22 October 2026, and Phase 3 (monitoring, drift detection, interactive demonstration and documentation hardening) on 16 November 2026. The defence takes place on 17 and 18 November 2026 and comprises a 15-minute architecture presentation, a 5-minute live demonstration and a 10-minute question-and-answer session.

In order to avoid any ambiguity between the target design and the present state of the code base, Table 9.1 records the status of each capability as verified against the repository. The label *implemented* denotes functionality that exists in the code base and is exercised by the test suite or the deployment manifests; *partial* denotes functionality whose building blocks exist but which is

not yet integrated end-to-end; and *planned* denotes functionality that is specified in this report but has not yet been implemented.

Table 9.1: Implementation status of principal capabilities, verified against the repository (`feature/pipeline1-monitoring` branch, November 2026).

Capability	**Status**	**Verified Repository Evidence & Operational Alignment**
Reproducible environment (Pixi, Just)	**implemented**	Lock-file driven `pixi.lock` environments (`default`, `dev`, `ci-dev`) and `Justfile` orchestration; zero unmanaged virtualenvs.
Dataset versioning & S3 streaming	**implemented**	DVC on DagsHub S3 remote (`dvc.lock`); `s3_streamer.py` with connection pooling, retries, and `min_extraction_success_ratio` gate.
Consolidated PostgreSQL 4-DB cluster	**implemented**	Single PostgreSQL 16 cluster (`postgres:16-alpine`) hosting isolated `mlflow`, `airflow`, `telemetry`, and `optuna` databases; non-CoW NVMe allocation (`chattr +C`); SQLite local fallback.
MLflow 3.x LoggedModels & 4-split Lineage	**implemented**	Self-hosted MLflow 3.16.0 proxy backed by PostgreSQL and SeaweedFS S3 (`s3://mlflow-artifacts/`); first-class `LoggedModel` (`m-...`), 4-split dataset lineage via `mlflow.data`, real tensor signatures, and cross-stage DVC run continuity via `reports/train_metrics.json:mlflow_run_id`.
Head training & convergence callbacks	**implemented**	Class-Balanced Loss (β), best-weight restoration, early stopping (patience 5), and LR plateau decay (patience 2) in `loops.py`.
Bayesian HPO (Optuna TPE + ASHA)	**implemented**	6D search space in `tuning.py` backed by PostgreSQL `optuna` database with `postgresql+psycopg2://` driver compatibility and nested MLflow child trial logging.
Dual-output ONNX & INT8 PTQ	**implemented**	`onnx_exporter.py` emits unnormalised `logits` ($[N, K]$) and penultimate `embeddings` ($[N, D]$); static QDQ and dynamic fallback with logit node preservation in `quantizer.py`.
Epistemic safety gating (OOD + Conformal)	**implemented**	Dual-gate OOD (dynamically calibrated τ_{OOD} resolved via `config/conformal_config.yaml`, $\tau_{\text{entropy}} = 1.60$ nats) in `ood_detector.py` and Split Conformal sets ($1 - \alpha \geq 0.95$, $\tau_{\text{conf}} = 0.60$) in `conformal.py`.
Inference service & HTMX dashboard	**implemented**	FastAPI service (`/predict`, `/explain`, `/health/ready`), distributed Redis ephemeral upload cache (`redis-ephemeral:6379`) eliminating multi-pod split-brain for CAM, ViT patch-grid projection and forward-hooked linear CAM saliency with differential candidate toggles (`inference/cam.py`), and Model Passport attestation.

Capability	**Status**	**Verified Repository Evidence & Operational Alignment**
Hybrid CI/CD & K3s GPU execution	**implemented**	GitHub Actions quality/release/GHCR workflows (`ci.yml`, `release.yaml`), Traefik v3.1 canary ingress, and bare-metal Jenkins two-pod execution (`jenkins-cpu-runner` and dedicated RTX 5070 Ti `jenkins-gpu-runner`).
Test harness & code quality gates	**implemented**	203 `pytest` tests, zero-warning Ruff, MyPy across 70 source files, 8-gate attestation seal (`attest_seal.py`), AST code invariants engine (`verify_code_invariants.py`), automated hardware specification verification, 100% cognitive complexity ≤ 15 with zero snapshot exemptions, and OKF v0.2 schema validation.
Two-tier drift detection (W_1 & MMD^2)	**implemented**	Linear-time Wasserstein-1 (W_1) on pre-normalised activations and full-geometry RBF MMD^2 with $B = 1{,}000$ Monte Carlo permutations in `monitoring/drift.py`; exposed via `/api/v1/drift/evaluate` and DVC Stage 3.5 (`data_drift.py`).
Stage 7 Four-Gate Promotion Verdict	**implemented**	Terminal DVC evaluation gate in `pipelines/evaluate.py` asserting Macro PR-AUC superiority, INT8 degradation $\leq 3\%$, conformal coverage $\geq (1-\alpha) - 5\%$, and latency SLO ≤ 120 ms with live MMD^2 exchangeability ($p \geq 0.01$); generates `reports/model_passport.json`.
In-cluster S3 cache & Optuna dashboard	**implemented**	Kubernetes manifests deploy SeaweedFS (Apache 2.0, port 8333) and Optuna dashboard (NodePort 30081) backed by PostgreSQL 16 persistence; `docker-compose.yml` mirrors the stack locally.
Prometheus & Grafana observability	**implemented**	FastAPI exposes `/metrics` with latency histograms, prediction counts, and embedding norms in `telemetry.py`; Prometheus server scrapes endpoints with native TSDB; Grafana alert rules route drift alerts to webhook relay.
Airflow 3 LocalExecutor orchestration	**planned**	Fully specified architecture in Chapter 4; `prediction_drift_monitor` DAG (@hourly) and event-driven `model_retraining` DAG with authenticated webhook relay and Jenkins trigger.
Evidently AI HTML dashboards in MLflow	**implemented**	Interactive Evidently drift reports and machine-readable metrics logged directly to MLflow experiment `taxon-vision-drift` in Stage 3.5 and drift endpoints.
BADGE active learning & triage queue	**partial**	`compute_badge_embeddings` (`query.py`) and Parquet-backed `TriageQueueManager` (`triage_queue.py`) are unit-tested, pending full integration with the live triage queue.
LanceDB filtered HNSW & RRF (Pipeline 2)	**planned**	Single-stage spatio-temporal HNSW index, Uber H3 masking, and Reciprocal Rank Fusion (Chapter 5) fully preserved as architectural pillars for continuous taxonomic MMKG.

Capability	**Status**	**Verified Repository Evidence & Operational Alignment**
Zero-shot cropping & VLM triage (Pipeline 2)	**planned**	Upstream YOLO-World organism isolation, `taxon-vision-vlm` (Qwen2-VL-2B) gRPC microservice, and DuckDB taxonomic split/lump mutations (Chapter 5).
Label Studio synchronisation	**planned**	Automated bidirectional sync between `triage_queue` and the DagsHub Label Studio annotation workspace.

With the agent pod GPU allocation, the CUDA 12.8 toolchain upgrade, the foundation vision feature extraction pipeline, and the 8-gate agentic attestation overhaul completed, the critical path moves from pipeline mechanics to scaling data horizons. Observations streamed from AWS S3 are encoded by the frozen DINOv3 backbone and cached to disk, enabling genuine classification convergence on biological taxa. Subsequent iterations will expand the reference manifest from the initial 518 pilot exemplars to regional partitions, implement scheduled drift evaluation, and activate the multimodal retrieval index.

Bibliography

[1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.

[2] Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse gradient embeddings. In *International Conference on Learning Representations (ICLR)*, 2020.

[3] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9268–9277. IEEE, 2019.

[4] Pengzhen Ren, Yun Xiao, Xiaojun Chang, Po-Yao Huang, Zhihui Li, Brij B. Gupta, Xiaojiang Chen, and Xinwang Wang. A survey of deep active learning. *ACM Computing Surveys (CSUR)*, 54(9):1–40, 2021.

[5] Samuel Stevens, Jiaman Wu, Matthew J. Thompson, Elizabeth G. Campolongo, Chan Hee Song, David E. Carlyn, Li Dong, Wasila M. Dahdul, Charles Stewart, Tanya Berger-Wolf, Wei-Lun Chao, and Yu Su. Bioclip: A vision foundation model for the tree of life. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19412–19424. IEEE, 2024.

[6] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The inaturalist species classification and detection dataset. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8769–8778. IEEE, 2018.